

A Universal Blinder: One-round Blind Signatures from FHE

Dan Boneh and Jaehyung Kim

Stanford University
`{dabo,jaehk}@cs.stanford.edu`

March 23, 2026

Abstract

We construct compilers that convert any secure signature scheme into a single-round blind signature scheme. An important property of the construction is that the final blind signature has exactly the same format as the underlying signature scheme, making the blind signature scheme backwards compatible with the underlying scheme. Our compilers make use of (two-key) fully homomorphic encryption and zero-knowledge proofs to ensure unforgeability and blindness of the final scheme. We present three compilers where the main differences is which party does the bulk of the work: the client, the signer, or both. Along the way we introduce a new notion of *verifiable FHE* that we call *committed verifiable FHE*, where the verifier does not see the circuit in the clear.

Contents

1	Introduction	3
2	Preliminaries	6
2.1	Signature schemes	6
2.2	Blind Signature	7
2.3	Fully Homomorphic Encryption (FHE)	8
2.4	Multikey FHE with Threshold Decryption	10
2.5	Non-interactive Zero Knowledge (NIZK)	12
2.6	Commitment	13
3	One-round Blind Signature from MKFHE	14
4	Committed Verifiable FHE	19
4.1	Definition	19
4.2	Construction	21
5	Blind Signatures with Little Server Side Computation	23
6	Extensions	25
6.1	Relaxing Perfect Correctness	25
6.2	Threshold Blind Signatures	27
A	Blind Signatures with Little Client Side Computation	31
B	Deferred Proofs	35

1 Introduction

A blind signature scheme [Cha82] is an important privacy mechanism used for digital cash, voting systems, and in privacy-preserving authorization systems [CDVW24]. Briefly, a blind signature is a two-party protocol between a client and a signer, where the client takes a message m as input and the signer takes a secret signing key sk as input. At the end of the protocol the client has a signature σ on m , and nothing else, while the signer learns nothing. The scheme should be one-more unforgeable and blind [PS96, Fis06]. Here, blindness means that when the final signature σ is presented, no one should be able to link it to the protocol instance where it was generated, not even a malicious signer who may have generated a malformed public key. We review the security definitions in Section 2.2. When the blind signature protocol is multi-round, one also requires that the scheme is secure under concurrent composition [BLL⁺22, FW24, CATZ24].

While efficient single-round blind signing protocols can be constructed for the RSA and BLS signature schemes [BNPS03, Bol03], for other signature schemes such as ECDSA, Schnorr, and lattice schemes, we only have blind variants that require a multi-round signing protocol (e.g., [QCY21, FW24, MPD25] to name a few). Many custom one-round blind signatures¹ have appeared in the literature, such as [FHS15, FHKS16, KNY21, KRS23].

Fischlin [Fis06] describes a universal blinder that compiles any signature scheme into a one-round blind signature scheme. However, the resulting signatures are very different from a signature in the underlying scheme. In Fischlin’s construction, the client sends a hiding commitment to the message m , the signer responds with a signature on the commitment, and the client generates a non-interactive zero-knowledge proof of knowledge that it has a signature on a commitment to m . The final signature is a zero-knowledge proof, not a signature in the underlying scheme. Another universal blinder was proposed by Garg et al. [GRS⁺11] using garbled circuits and ZAPs, which results in a large amount of traffic between the client and the signer.

Our contributions. Boneh and Kim [BK25] recently informally described an FHE-based construction for a universal blinder that has (i) a one-round signing protocol, and (ii) the final signature is the same as in the underlying scheme. Similar schemes were also proposed by Schröder and Unruh [SU11] and Agrawal et al. [AKSY21]. In this paper we flesh out, formalize, and generalize this approach.

Let us first consider a strawman construction and its limitations. Let $SIG = (\text{KeyGen}, \text{Sign}, \text{Verify})$ be a (non-blind) signature scheme and let (pk_{SIG}, sk_{SIG}) be a key pair for this scheme. The strawman blind signing protocol for SIG works as follows:

- To request the signature on a message m , the client first generates an FHE key pair (pk_{FHE}, sk_{FHE}) , encrypts m as $c \leftarrow \text{FHE.Encrypt}(pk_{FHE}, m)$, and sends (c, pk_{FHE}) to the signer. Semantic (IND-CPA) security of the FHE scheme ensures that the signer learns nothing about m .
- Next, the signer runs $c' \leftarrow \text{FHE.Eval}(pk_{FHE}, f, c)$ where the function f is defined as $f(m) := \text{SIG.Sign}(sk_{SIG}, m)$, so that c' is an encryption of $f(m)$. The signer sends c' back to the client.
- Finally, the client decrypts c' to get $\sigma \leftarrow \text{FHE.Dec}(sk_{FHE}, c')$.

This completes the description of the one-round blind signing protocol. If both parties are honest, then the final signature is indeed a well formed signature on m with respect

¹These schemes are sometimes called *round-optimal schemes*, however we use the term *one-round schemes* because it is more descriptive, and because it has been shown that round-optimal schemes are not round optimal [Han23].

to the signature scheme SIG. Even when both parties are honest we must require that the FHE scheme be *circuit private*, meaning that c' should reveal nothing about the function f beyond the fact that $f(m) = \sigma$. This ensures that an honestly generated c' does not leak information about sk_{SIG} .

At first glance it may appear that the client obtains a signature on m while the signer learns nothing on about m , thereby satisfying the required blindness properties. However, there are several ways in which this strawman scheme can fail and become insecure.

First, a malicious client may compromise one-more unforgeability by learning information about the signer's secret key sk_{SIG} . To do so the client can generate a malformed FHE public key pk_{FHE} and a malformed FHE ciphertext c , so that when the signer sends back c' , the client learns the signer's secret key sk_{SIG} . Consequently, the signer must somehow ensure that pk_{FHE} and c are well formed.

Second, a malicious server may compromise the blindness properties of the scheme. For example, if the signing algorithm $\text{Sign}(\text{sk}_{\text{SIG}}, m; r)$ is randomized, the server could use one distribution on r in one session, and a different distribution on r for another session. This might let it link the final signature to the session that generated it. Even worse, the signature scheme may be such that there are two valid secret keys sk_{SIG} and sk'_{SIG} that correspond to pk_{SIG} . The server could use sk_{SIG} in one session and sk'_{SIG} in another session, letting it link the final signature to the session that generated it. Moreover, we must also ensure that blindness holds even if the server uses a malformed public key pk_{SIG} .

To address all these issues, we describe three secure compilers that extend the strawman FHE scheme. The full constructions, along with their proofs of security, are presented in the main body.

Compiler 1: both sides compute an encrypted signature. Our first construction makes use of two-key FHE, which is a special case of multi-key FHE [MW16, KKL+23, CDKS19]. To review the concept of two-key FHE consider two ciphertexts c_1, c_2 that are encryptions under different public keys: $c_1 \leftarrow \text{FHE.Encrypt}(\text{pk}_{\text{FHE}}^{\text{CLNT}}, m_1)$ and $c_2 \leftarrow \text{FHE.Encrypt}(\text{pk}_{\text{FHE}}^{\text{SCNR}}, m_2)$. Then for a function $f : \mathcal{M}^2 \rightarrow \mathcal{M}$ that is computable by a poly-size circuit, one can compute

$$c' \leftarrow \text{FHE.Eval}((\text{pk}_{\text{FHE}}^{\text{CLNT}}, \text{pk}_{\text{FHE}}^{\text{SCNR}}), f, c_1, c_2)$$

to obtain an encryption of $f(m_1, m_2)$. However, to decrypt c' one first partially decrypts c' using each of the secret keys $\text{sk}_{\text{FHE}}^{\text{CLNT}}$ and $\text{sk}_{\text{FHE}}^{\text{SCNR}}$ to obtain two partial decryptions. The two partial decryptions are then combined to obtain the full decryption of c' . Our blind signature scheme does not require that FHE.Eval is circuit private. However, we do require that it be a deterministic algorithm.

From here on we will assume, without loss of generality, that the signature scheme SIG has a *deterministic* signing algorithm. That is, $\text{SIG.Sign}(\text{sk}_{\text{SIG}}, m)$ is deterministic. The reason we can make this assumption is that any signature scheme can be de-randomized by using a secure pseudorandom function to generate the random bits used by the signing algorithm (see [BS23, Ex. 13.6]). The resulting signature scheme need not be unique, that is, there can be many valid signatures for a given message and public key.

With these tools in place, the blind signature scheme derived from SIG works as follows (see Section 3 for the details). First, during key generation, the signer does: (i) generate a SIG key pair $(\text{pk}_{\text{SIG}}, \text{sk}_{\text{SIG}})$ and an FHE key pair $(\text{pk}_{\text{FHE}}^{\text{SCNR}}, \text{sk}_{\text{FHE}}^{\text{SCNR}})$, (ii) compute $c_s \leftarrow \text{FHE.Encrypt}(\text{pk}_{\text{FHE}}^{\text{SCNR}}, \text{sk}_{\text{SIG}})$, (iii) compute a commitment com to $(\text{sk}_{\text{SIG}}, \text{sk}_{\text{FHE}}^{\text{SCNR}})$, and (iv) output the public key $\text{pk}'_{\text{SIG}} = (\text{pk}_{\text{SIG}}, c_s, \text{com}, \pi)$. Here π is a non-interactive zero-knowledge proof (NIZK) that $(\text{pk}_{\text{SIG}}, \text{sk}_{\text{SIG}})$ and $(\text{pk}_{\text{FHE}}^{\text{SCNR}}, \text{sk}_{\text{FHE}}^{\text{SCNR}})$ are well-formed key pairs, and that com and c_s were computed correctly. The full pk'_{SIG} is only used in the blind

signing protocol, while signature verification only uses pk_{SIG} as in the underlying signature scheme SIG.

Next, to generate a signature on a message m the client checks the proof π in pk'_{SIG} and if valid does (i) generate an FHE key pair $(\text{pk}_{\text{FHE}}^{\text{CLNT}}, \text{sk}_{\text{FHE}}^{\text{CLNT}})$, (ii) compute $c_c \leftarrow \text{FHE.Encrypt}(\text{pk}_{\text{FHE}}^{\text{CLNT}}, m)$, and (iii) send $(\text{pk}_{\text{FHE}}^{\text{CLNT}}, c_c, \pi')$ to the signer, where π' is a NIZK that pk_{SIG} is a well-formed public key, and c_c is a well-formed ciphertext. Here it is important that the NIZK proof system is knowledge sound and straight-line extractable. The security proof crucially depends on these properties.

The signer checks the proof π' and if valid it homomorphically computes a two-key FHE ciphertext $c \leftarrow \text{FHE.Eval}((\text{pk}_{\text{FHE}}^{\text{CLNT}}, \text{pk}_{\text{FHE}}^{\text{SCNR}}), f, c_s, c_c)$ where the public function f is defined as $f(\text{sk}, m) := \text{SIG.Sign}(\text{sk}, m)$. It next partially decrypts c using $\text{sk}_{\text{FHE}}^{\text{SCNR}}$ and sends the partial decryption c' back to the client, along with a proof π'' that partial decryption was computed correctly, where the public statement is (c, com, c_s) . We stress that there is no need to prove that the homomorphic evaluation was done correctly.

Finally, the client uses c_c and c_s to compute the same ciphertext c on its own, namely $c \leftarrow \text{FHE.Eval}((\text{pk}_{\text{FHE}}^{\text{CLNT}}, \text{pk}_{\text{FHE}}^{\text{SCNR}}), f, c_c, c_s)$. It then checks the proof π'' , and if valid it uses $\text{sk}_{\text{FHE}}^{\text{CLNT}}$ to compute its own a partial decryption of c , and combines that partial decryption with c' from the signer to obtain the final signature σ .

This completes the informal description of the scheme. All the NIZK proofs computed in this process attest to the validity of correct key generation, encryption, and partial decryption. These statements are relatively simple meaning that these proofs can be efficiently generated. The main bottleneck in the scheme comes from the fact that both the client and the signer need to execute SIG.Sign homomorphically on the encrypted message and encrypted secret key. For an algorithm like ECDSA.Sign there are custom FHE schemes optimized for this task [BK25].

To prove security we rely on the straight-line extraction property of the NIZK. We also require that the FHE scheme has perfect correctness, meaning that there are no decryption errors. In Section 6 we show how to relax this requirement.

Compilers 2 and 3. Our two other compilers, described in Section 5 and Appendix A, avoid the need for both the client and the signer to homomorphically compute the encrypted signature. Only one of the parties needs to do so. The downside of both compilers is the need for a more complicated zero-knowledge proof. In Section 5 only the client homomorphically computes the signature, while the signer just checks a proof and decrypts. This scheme is well suited for a setting where the signer is a slow high-security device. In contrast, in the scheme in Appendix A only the signer homomorphically computes the signature, as in the strawman construction above. This signer-only scheme is related to a blind signature construction by Schröder and Unruh [SU11] and Agrawal et al. [AKSY21].

Because only one side is computing the encrypted signature, that side must prove that it correctly carried out the entire homomorphic computation. To do so, both client-side and signer-side schemes, require a strong notion of verifiable FHE [KVMH24, ABPS24] that we call *committed verifiable FHE*, or CVFHE. Recall that in verifiable FHE, the party that does the homomorphic evaluation proves that evaluation was done correctly with respect to a *public* function f . In our one-sided schemes, the evaluation is done with respect to a secret function, or more precisely, with respect of a function that is defined by a secret value: in Appendix A the secret value is the secret (signing) key, while in Section 5 the secret value is the secret message. To do so we define CVFHE which lets the evaluator prove correct evaluation with respect to a *committed* function. CVFHE naturally comes up in other contexts, such as verifiable inference on private queries with respect to a committed machine learning model.

We conclude in Section 6 with a number of extensions of our techniques, such as a universal one-round *threshold* blind signature compiler.

Additional related works A *universal thresholdizer* [BGG⁺18, EY24] uses FHE to compile any signature scheme into a threshold signature scheme. One can view our universal blinder as a complementary compiler for blind signatures. In Section 6 we combine the two compilers to obtain a universal compiler for threshold blind signatures.

Non-interactive blind signature (NIBS) [Han23] allows the signer to generate a pre-signature using the user’s public key, and later the user can obtain a signature using the pre-signature. This can be regarded as obtaining a signature without communication, which is why it is said to be non-interactive. Hanzlik [Han23] builds a generic compiler using proof systems, while subsequent work [BCGY24] uses circuit private LHE and NIZK to compile any signature scheme into a NIBS.

Verifiable FHE [KVMH24, ABPS24] guarantees integrity against a malicious server. Most of the works in the literature assumes an evaluation of a public function, where attaching an evaluation proof for the homomorphic circuit is sufficient to achieve verifiability. In many applications (as described above), we often need private evaluation such as private input from the server, or the circuit itself is private. Vinad et al. [VKH23] define a verifiable FHE with server privacy, proving the existence of a witness w (which is a server input) that satisfies $f(x, w) = y$ for a function f , a client input x , and the output y . Our CVFHE definition extends this definition to proving correct evaluation with respect to a committed circuit.

2 Preliminaries

For an integer k , we let $[k] = \{1, 2, \dots, k\}$. We denote the adversary by $\mathcal{A}$, the challenger by $\mathcal{C}$, and the scheme’s message space by $\mathcal{M}$. A function $f(\lambda)$ is **poly**(λ) if there exists a $c \in \mathbb{N}$ such that $f(\lambda) = O(\lambda^c)$. If $f(\lambda) = o(\lambda^{-c})$ for all $c \in \mathbb{N}$, we say that $f(\lambda)$ is **negligible** and write $f \leq \text{negl}(\lambda)$. A probability that is $1 - \text{negl}(\lambda)$ is **overwhelming**. We write $x \leftarrow A(y)$ to denote the assignment of the output of algorithm A on input y to x . When A is a randomized algorithm, then x is a random variable. For a randomized algorithm A we write $A(y; r)$ to denote the (deterministic) output of A on input y with randomness r .

2.1 Signature schemes

We use the standard definition of a signature scheme, except that we require a deterministic signing algorithm. Any randomized signing algorithm can be made deterministic using a secure PRF [BS23, Ex. 13.6].

Definition 2.1 (Signature scheme). A deterministic signature scheme is a 4-tuple of PPT algorithms $S = (\text{Setup}, \text{KeyGen}, \text{Sign}, \text{Verify})$ where:

- $\text{Setup}(1^\lambda) \rightarrow \text{pp}$: for a security parameter λ , outputs public parameters pp .
- $\text{KeyGen}(1^\lambda, \text{pp}) \rightarrow (\text{pk}, \text{sk})$: outputs a public key and a secret key.
- $\text{Sign}(1^\lambda, \text{sk}, m) \rightarrow \sigma$: a **deterministic algorithm** that signs m using sk .
- $\text{Verify}(1^\lambda, \text{pk}, m, \sigma) \rightarrow \{0, 1\}$: verifies a signature on a message m .

We will frequently drop the 1^λ argument to all but the Setup algorithm.

Definition 2.2 (Correctness). A signature scheme is **correct** if for all m in the message space we have

$$\Pr[\text{Verify}(\text{pk}, m, \text{Sign}(\text{sk}, m)) = 1] \geq 1 - \text{negl}(\lambda),$$

where $\text{pp} \leftarrow \text{Setup}(1^\lambda)$ and $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\text{pp})$.

Definition 2.3 (EUF-CMA). We say that the scheme is **secure** if the advantage of every PPT adversary $\mathcal{A}$ in the following game is a negligible function of λ :

- The challenger runs $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\text{pp})$ and sends pk to $\mathcal{A}$.
- $\mathcal{A}$ queries the challenger (polynomially) many times. Let m_i be query number i . The challenger computes $\sigma_i \leftarrow \text{Sign}(\text{sk}, m_i)$ and sends σ_i to $\mathcal{A}$.
- $\mathcal{A}$ outputs a pair (m, σ) .
- The adversary's advantage is defined as

$$\Pr[\text{Verify}(\text{pk}, m, \sigma) = 1 \wedge m \notin \{m_1, m_2, \dots\}].$$

2.2 Blind Signature

We adapt the definition of blind signature from [CATZ24] to a single round setting.

Definition 2.4 (One-round blind signature). A one-round blind signature scheme BS is a tuple of PPT algorithms

$$\text{BS} = (\text{Setup}, \text{KeyGen}, \text{Signer}, \text{User}_1, \text{User}_2, \text{Verify})$$

where:

- $\text{Setup}(1^\lambda) \rightarrow \text{pp}$: for a security parameter λ , outputs public parameters pp .
- $\text{KeyGen}(1^\lambda, \text{pp}) \rightarrow (\text{pk}, \text{sk})$: outputs a public key and a secret key.
- Signing Protocol: a one-round protocol between the signer and the user to sign on a message m blindly. The protocol is implemented by three algorithms User_1 , Signer , User_2 as follows

$$\begin{aligned} (\text{st}, \text{umsg}) &\leftarrow \text{User}_1(1^\lambda, \text{pk}, m) \\ \text{smsg} &\leftarrow \text{Signer}(1^\lambda, \text{sk}, \text{umsg}) \\ \sigma &\leftarrow \text{User}_2(1^\lambda, \text{st}, \text{smsg}) \end{aligned}$$

Here, the final output σ is either a signature or $\perp$ indicating an error.

- $\text{Verify}(1^\lambda, \text{pk}, m, \sigma) \rightarrow \{0, 1\}$: verifies a signature on a message m .

As usual, we will often drop the 1^λ argument to all but the Setup algorithm.

Definition 2.5 (Correctness). We say that a blind signature BS is **correct** if for all m in the message space we have

$$\Pr[\text{Verify}(\text{pk}, m, \sigma) = 1] \geq 1 - \text{negl}(\lambda),$$

where $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\text{pp})$, $\text{pp} \leftarrow \text{Setup}(1^\lambda)$, and σ is the result of the signing protocol applied to pk, sk, m .

Games $\mathbf{G}_{\text{BS},\mathcal{A}}^{\text{OMUF}}(\lambda)$ and $\mathbf{G}_{\text{BS},\mathcal{A}}^{\text{OMSUF}}(\lambda)$	Oracle $\text{OSign}(\text{umsg})$
1 : $\text{pp} \leftarrow \text{BS.Setup}(1^\lambda), (\text{pk}, \text{sk}) \leftarrow \text{BS.KeyGen}(\text{pp})$	1 : $\ell \leftarrow \ell + 1$
2 : $\ell \leftarrow 0$	2 : $\text{smsg} \leftarrow \text{BS.Signer}(\text{sk}, \text{umsg})$
3 : $\{(m_k^*, \sigma_k^*)\}_{k \in [\ell+1]} \leftarrow \mathcal{A}^{\text{OSign}(\cdot)}(\text{pp}, \text{pk})$	3 : return smsg
4 : if $\exists k \in [\ell+1] : \text{BS.Verify}(\text{pk}, m_k^*, \sigma_k^*) = 0$ return 0	
5 : $\mathbf{G}_{\text{BS},\mathcal{A}}^{\text{OMUF}} : \text{if } \exists k_1 \neq k_2 : m_{k_1}^* = m_{k_2}^* \text{ return } 0$	
6 : $\mathbf{G}_{\text{BS},\mathcal{A}}^{\text{OMSUF}} : \text{if } \exists k_1 \neq k_2 : (m_{k_1}^*, \sigma_{k_1}^*) = (m_{k_2}^*, \sigma_{k_2}^*) \text{ return } 0$	
7 : return 1	

Figure 1: One-more unforgeability for a blind signature scheme BS and adversary $\mathcal{A}$.

Unforgeability. For signature security, we require that the scheme satisfy a notion called *one-more unforgeability* [BNPS02] or simply OMUF. Here the adversary can make ℓ blind signing queries and is required to produce $\ell + 1$ valid message-signature pairs, for $\ell + 1$ distinct messages. For strong security, this notion is denoted by OMSUF, where the adversary is only required to produce $\ell + 1$ distinct valid message-signature pairs (meaning that generating multiple distinct signatures for the same message is an acceptable forgery by the adversary).

Definition 2.6 (One-more unforgeability). We say that a blind signature scheme BS is **secure** if for every PPT adversary $\mathcal{A}$ we have

$$\text{Adv}_{\text{BS},\mathcal{A}}^{\text{OMUF}}(\lambda) := \Pr \left[\mathbf{G}_{\text{BS},\mathcal{A}}^{\text{OMUF}}(\lambda) = 1 \right] < \text{negl}(\lambda)$$

for the Game $\mathbf{G}_{\text{BS},\mathcal{A}}^{\text{OMUF}}$ in Figure 1. The scheme is **strongly secure** if the same holds for the Game $\mathbf{G}_{\text{BS},\mathcal{A}}^{\text{OMSUF}}$ in Figure 1.

Blindness. For blindness, we require that no signature can be linked to an instance of the signing protocol that generated that signature. This should hold even against an adversary that knows the secret signing key or generates a malformed public key. Briefly, Definition 2.7 below requires that an adversary that provides a (possibly malformed) public key pk and two messages m_0, m_1 cannot distinguish between the following two distributions

$$\{(\text{pk}, m_0, m_1, \text{umsg}_0, \text{umsg}_1, \sigma_0, \sigma_1)\} \quad \text{and} \quad \{(\text{pk}, m_0, m_1, \text{umsg}_1, \text{umsg}_0, \sigma_0, \sigma_1)\}$$

where $(\text{st}_i, \text{umsg}_i) \leftarrow \text{BS.User}_1(\text{pk}, m_i)$ for $i = 0, 1$. This also implies blindness for more than two messages.

Definition 2.7 (Blindness). We say that a blind signature scheme BS is **blind** if for every PPT adversary $\mathcal{A}$ in Game $\mathbf{G}_{\text{BS},\mathcal{A}}^{\text{blind}}$ in Figure 2 we have

$$\text{Adv}_{\text{BS},\mathcal{A}}^{\text{blind}}(\lambda) := \left| \Pr \left[\mathbf{G}_{\text{BS},\mathcal{A}}^{\text{blind}}(\lambda) = 1 \right] - \frac{1}{2} \right| < \text{negl}(\lambda)$$

2.3 Fully Homomorphic Encryption (FHE)

We consider a standard fully homomorphic encryption (FHE) with circuit privacy. We adapt the threshold FHE definition of [BGG⁺18] to a single-party case.

Game $\mathbf{G}_{\text{BS}, \mathcal{A}}^{\text{blind}}(\lambda)$	Oracle $\text{OU}_1(i)$
1 : $\text{pp} \leftarrow \text{BS.Setup}(1^\lambda)$	1 : if $i \notin \{0, 1\}$ OR $\text{sess}_i \neq 1$: return $\perp$
2 : $b \leftarrow \$ \{0, 1\}$	2 : $\text{sess}_i \leftarrow 2$ // move session to phase 2
3 : $b_0 \leftarrow b, \quad b_1 \leftarrow 1 - b$	3 : $(\text{st}_i, \text{umsg}_i) \leftarrow \text{BS.User}_1(\text{pk}, m_{b_i})$
4 : $b' \leftarrow \mathcal{A}^{\text{INIT}, \text{OU}_1, \text{OU}_2}(\text{pp})$	4 : return umsg_i
5 : return $b' = b$	
Oracle $\text{INIT}(\text{pk}', m'_0, m'_1)$	Oracle $\text{OU}_2(i, \text{smsg})$
1 : $\text{sess}_0, \text{sess}_1 \leftarrow 1$	1 : if $i \notin \{0, 1\}$ OR $\text{sess}_i \neq 2$: return $\perp$
2 : $\text{pk} \leftarrow \text{pk}'$	2 : $\text{sess}_i \leftarrow 3$ // mark session as finished
3 : $m_0 \leftarrow m'_0, \quad m_1 \leftarrow m'_1$	3 : $\sigma_{b_i} \leftarrow \text{BS.User}_2(\text{st}_i, \text{smsg})$
	4 : if $\text{sess}_0 = \text{sess}_1 = 3$: // both sessions finished
	5 : if $\sigma_0 \neq \perp$ AND $\sigma_1 \neq \perp$: return (σ_0, σ_1)
	6 : else return $(\perp, \perp)$
	7 : return (i, closed) // only session i finished

Figure 2: The blindness game for a blind signature scheme BS and adversary $\mathcal{A}$.

Definition 2.8 (Fully Homomorphic Encryption). A fully homomorphic encryption scheme is a tuple of PPT algorithms

$$\text{FHE} = (\text{Setup}, \text{KeyGen}, \text{Encrypt}, \text{Eval}, \text{ReRand}, \text{Decrypt})$$

such that:

$\text{Setup}(1^\lambda) \rightarrow \text{pp}$: Given a security parameter λ , the parameter generation algorithm **Setup** outputs a string of public parameters pp which is a default input to all other algorithms.

$\text{KeyGen}(\text{pp}) \rightarrow (\text{pk}, \text{sk})$: On input a public parameter pp , the key generation algorithm outputs a public key pk and a secret key sk .

$\text{Encrypt}(\text{pk}, m) \rightarrow \text{ct}$: On input a public key pk and a message $m \in \mathcal{M}$, the encryption algorithm outputs a ciphertext ct .

$\text{Eval}(\text{pk}, C, \text{ct}_1, \dots, \text{ct}_k) \rightarrow \text{ct}'$: On input a public key pk , a circuit $C : \mathcal{M}^k \rightarrow \mathcal{M}$, and ciphertexts $\text{ct}_1, \dots, \text{ct}_k$, the **deterministic** evaluation algorithm outputs a ciphertext ct' .

$\text{ReRand}(\text{ct}; r) \rightarrow \text{ct}'$: On input a ciphertext ct and a randomness r , the rerandomization algorithm outputs a ciphertext ct' that decrypts to the same message.²

$\text{Decrypt}(\text{sk}, \text{ct}) \rightarrow m$: On input a secret key sk and a ciphertext ct , the decryption algorithm outputs a message $m \in \mathcal{M}$.

We expect that an FHE scheme satisfies the following properties: Perfect Correctness, IND-CPA Security, and Circuit Privacy.

²When the randomness r is omitted, it means that we randomly choose r from the randomness distribution.

Definition 2.9 (Perfect Correctness). For a security parameter λ , a circuit $C : \mathcal{M}^k \rightarrow \mathcal{M}$, messages $m_j \in \mathcal{M}$ for each $j \in [k]$, and using $\text{pp} \leftarrow \text{Setup}(1^\lambda)$, it holds that:

$$\Pr \left[\begin{array}{c|c} \text{Decrypt}(\text{sk}, \text{ct}') & \begin{array}{l} (\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\text{pp}) \\ \forall j \in [k], \text{ct}_j \leftarrow \text{Encrypt}(\text{pk}, m_j) \\ \text{ct}' \leftarrow \text{Eval}(\text{pk}, C, \text{ct}_1, \dots, \text{ct}_k) \\ \text{ct}' \leftarrow \text{ReRand}(\text{ct}') \end{array} \\ \hline = C(m_1, \dots, m_k) \end{array} \right] = 1.$$

Although the most efficient schemes rely on a weaker notion of correctness that allows a negligible failure probability, most of them can be made perfectly correct with reasonable overhead, as discussed in [Hal17, Definition 5.2.2]. We revisit the issue of perfect correctness in Section 6.

Definition 2.10 (IND-CPA Security). For a security parameter λ , for any PPT adversary $\mathcal{A}$, and using $\text{pp} \leftarrow \text{Setup}(1^\lambda)$, the adversary in the following attack game has a negligible advantage.

1. $\mathcal{C}$ runs $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\text{pp})$ and provides pk to $\mathcal{A}$.
2. $\mathcal{C}$ chooses $b \leftarrow \{0, 1\}$ and put $b_0 \leftarrow b$ and $b_1 \leftarrow 1 - b$.
3. Upon each query from $\mathcal{A}$,
 - (a) $\mathcal{A}$ chooses and sends $m_0, m_1 \in \mathcal{M}$ to $\mathcal{C}$.
 - (b) $\mathcal{C}$ provides $\text{Encrypt}(\text{pk}, m_{b_0})$, $\text{Encrypt}(\text{pk}, m_{b_1})$ to $\mathcal{A}$.³
4. $\mathcal{A}$ outputs a guess b' . The experiment outputs 1 if $b = b'$.

In this game, the advantage is defined as $|\Pr[b' = b] - 1/2|$.

Definition 2.11 (Circuit Privacy). Given a security parameter λ and using $\text{pp} \leftarrow \text{Setup}(1^\lambda)$, let $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\text{pp})$, $m_1, \dots, m_k \in \mathcal{M}$, and $C : \mathcal{M}^k \rightarrow \mathcal{M}$ be a circuit. Let $\text{ct}_i \leftarrow \text{Enc}(\text{pk}, m_i)$ for each $i \in [k]$ and $\text{ct} \leftarrow \text{Eval}(\text{pk}, C, \text{ct}_1, \dots, \text{ct}_k)$. We say that FHE satisfies **circuit privacy** if there exists a PPT simulator $\mathcal{S}$ such that

$$(\text{ReRand}(\text{ct}), \text{ct}_1, \dots, \text{ct}_k, \text{sk}, \text{pk}, 1^\lambda) \approx_s (\mathcal{S}(\text{pk}, C(m_1, \dots, m_k), \text{ct}_1, \dots, \text{ct}_k), \text{ct}_1, \dots, \text{ct}_k, \text{sk}, \text{pk}, 1^\lambda)$$

where $\approx_s$ denotes statistical indistinguishability.

2.4 Multikey FHE with Threshold Decryption

We consider n party multikey FHE with threshold decryption [MW16]. We adapt the definitions of [MW16, BGG⁺18]. The main difference between [MW16] and ours is that we discuss fully (not leveled) homomorphic encryption.

Definition 2.12 (Multikey FHE). Let $P = \{P_1, \dots, P_n\}$ be a set of parties. A multikey fully homomorphic encryption scheme with threshold decryption is a tuple of PPT algorithms

$$\text{MKFHE} = (\text{Setup}, \text{KeyGen}, \text{Encrypt}, \text{Expand}, \text{Eval}, \text{PartDec}, \text{FinDec})$$

such that:

³We deviate from the standard definition and provide two encryptions. The advantage of this game is at least half of the standard game with one encryption.

Setup(1^λ) $\rightarrow$ **pp**: Given a security parameter λ , the parameter generation algorithm **Setup** outputs a string of public parameters **pp** which is a default input to all other algorithms.

$(\mathbf{pk}, \mathbf{sk}) \leftarrow \mathbf{KeyGen}(\mathbf{pp})$: On input a public parameter **pp**, the key generation algorithm outputs a public key **pk** and a secret key **sk**. Each party P_i independently runs the key generation algorithm.

Encrypt($\mathbf{pk}, m$) $\rightarrow$ **ct**: On input a public key **pk** and a message $m \in \mathcal{M}$, the encryption algorithm outputs a ciphertext **ct**.

Expand($\hat{\mathbf{pk}}, i, \mathbf{ct}$) $\rightarrow$ $\hat{\mathbf{ct}}$: Input a tuple of public keys $\hat{\mathbf{pk}} = (\mathbf{pk}_1, \dots, \mathbf{pk}_n)$, an index $i \in [n]$, and a fresh ciphertext **ct** encrypted under $\mathbf{pk}_i$, the expansion algorithm outputs an expanded ciphertext $\hat{\mathbf{ct}}$.

Eval($\hat{\mathbf{pk}}, C, \hat{\mathbf{ct}}_1, \dots, \hat{\mathbf{ct}}_k$) $\rightarrow$ $\hat{\mathbf{ct}}$: On input a tuple of public keys $\hat{\mathbf{pk}} = (\mathbf{pk}_1, \dots, \mathbf{pk}_n)$, a circuit $C : \mathcal{M}^k \rightarrow \mathcal{M}$, and expanded ciphertexts $\hat{\mathbf{ct}}_1, \dots, \hat{\mathbf{ct}}_k$, the **deterministic** evaluation algorithm outputs a ciphertext $\hat{\mathbf{ct}}$.

PartDec($\mathbf{sk}_i, \hat{\mathbf{ct}}$) $\rightarrow$ p_i : On input a secret key $\mathbf{sk}_i$ and an expanded ciphertext **ct**, the partial decryption algorithm outputs a partial decryption p_i .

FinDec($p_1, \dots, p_n$) $\rightarrow$ $\hat{m}$: On input a list of partial decryption shares $p_1, \dots, p_n$, the final decryption algorithm outputs a plaintext $\hat{m} \in \mathcal{M}$.

We expect that a multikey FHE scheme with threshold decryption satisfies the following properties: Correctness, IND-CPA Security, and Simulation Security.

Definition 2.13 (Perfect Correctness). For a security parameter λ , a circuit $C : \mathcal{M}^k \rightarrow \mathcal{M}$, indices $i_j \in [n]$ and messages $m_j \in \mathcal{M}$ for each $j \in [k]$, and using $\mathbf{pp} \leftarrow \mathbf{Setup}(1^\lambda)$, it holds that:

$$\Pr \left[\begin{array}{l} \mathbf{FinDec}((\mathbf{PartDec}(\mathbf{sk}_{i_j}, \mathbf{ct}_j))_{j \in [k]}) \\ \quad = C(m_1, \dots, m_k) \end{array} \middle| \begin{array}{l} \forall i \in [n], (\mathbf{sk}_i, \mathbf{pk}_i) \leftarrow \mathbf{KeyGen}(\mathbf{pp}) \\ \hat{\mathbf{pk}} = (\mathbf{pk}_1, \dots, \mathbf{pk}_n) \\ \forall j \in [k], \mathbf{ct}_j \leftarrow \mathbf{Encrypt}(\mathbf{pk}_{i_j}, m_j) \\ \forall j \in [k], \hat{\mathbf{ct}}_j \leftarrow \mathbf{Expand}(\hat{\mathbf{pk}}, i_j, \mathbf{ct}_j) \\ \hat{\mathbf{ct}} \leftarrow \mathbf{Eval}(\hat{\mathbf{pk}}, C, \hat{\mathbf{ct}}_1, \dots, \hat{\mathbf{ct}}_k) \end{array} \right] = 1$$

Definition 2.14 (IND-CPA Security). For a security parameter λ , for any PPT adversary $\mathcal{A}$, and using $\mathbf{pp} \leftarrow \mathbf{Setup}(1^\lambda)$, the adversary in the following attack game has a negligible advantage.

1. $\mathcal{C}$ runs $(\mathbf{pk}, \mathbf{sk}) \leftarrow \mathbf{KeyGen}(\mathbf{pp})$ and provides **pk** to $\mathcal{A}$.
2. $\mathcal{C}$ chooses $b \leftarrow \{0, 1\}$ and put $b_0 \leftarrow b$ and $b_1 \leftarrow 1 - b$.
3. Upon each query from $\mathcal{A}$,
 - (a) $\mathcal{A}$ chooses and sends $m_0, m_1 \in \mathcal{M}$ to $\mathcal{C}$.
 - (b) $\mathcal{C}$ provides $\mathbf{Encrypt}(\mathbf{pk}, m_{b_0}), \mathbf{Encrypt}(\mathbf{pk}, m_{b_1})$ to $\mathcal{A}$.
4. $\mathcal{A}$ outputs a guess b' . The experiment outputs 1 if $b = b'$.

In this game, the advantage is defined as $|\Pr[b' = b] - 1/2|$.

We use the simulatable partial decryption definition in [MW16, Definition 4.2]. We slightly modify the definition to allow key generation with malicious randomness.

Definition 2.15 (Simulation Security). Given a security parameter λ and using $\text{pp} \leftarrow \text{Setup}(1^\lambda)$, let $(\text{pk}_i, \text{sk}_i) \leftarrow \text{KeyGen}(\text{pp})$ ⁴ for each $i \in [n]$, $m_1, \dots, m_k \in \mathcal{M}$, and $i_1, \dots, i_k \in [n]$. Let

$$\text{ct}_j \leftarrow \text{Encrypt}(\text{pk}_{i_j}, m_j), \text{ and } \hat{\text{ct}}_j \leftarrow \text{Expand}(\hat{\text{pk}}, i_j, \text{ct}_j)$$

for each $j \in [k]$. Let $C : \mathcal{M}^k \rightarrow \mathcal{M}$ be a circuit, and

$$\hat{\text{ct}} \leftarrow \text{Eval}(\hat{\text{pk}}, C, \hat{\text{ct}}_1, \dots, \hat{\text{ct}}_k)$$

be the evaluated ciphertext. We say that MKFHE satisfies simulation security if there exists a PPT algorithm $\mathcal{S}$ such that

$$(p_{i^*}, \text{pk}_1, \dots, \text{pk}_n, \text{ct}_1, \dots, \text{ct}_k, C, \text{sk}_{i^*}, \text{pp}, 1^\lambda) \approx_{\mathcal{S}} (\mathcal{S}(C(m_1, \dots, m_k), \hat{\text{ct}}, i^*, \{\text{sk}_i\}_{i \in [n] \setminus i^*}), \text{pk}_1, \dots, \text{pk}_n, \text{ct}_1, \dots, \text{ct}_k, C, \text{sk}_{i^*}, \text{pp}, 1^\lambda)$$

for all $i^* \in [n]$, where $p_{i^*} \leftarrow \text{PartDec}(\text{sk}_{i^*}, \hat{\text{ct}})$.

2.5 Non-interactive Zero Knowledge (NIZK)

We rephrase the definitions from [GOS06] and [CGSY24, Section 3.1].

Definition 2.16 (NIZK). Let L be a language with relation R . A non-interactive zero-knowledge (NIZK) is a tuple of PPT algorithms

$$\text{NIZK} = (\text{Setup}, \text{Prove}, \text{Verify})$$

such that:

Setup $(1^\lambda) \rightarrow \text{crs}$: On input a security parameter λ , the setup algorithm outputs a common reference string crs .

Prove $(\text{crs}, x, w) \rightarrow \pi$: On input a common reference string, a statement x , and a witness w , the proving algorithm **Prove** outputs a proof π .

Verify $(\text{crs}, x, \pi) \rightarrow b$: On input a common reference string crs , a statement x , and a proof π , the verification algorithm **Verify** outputs a verification bit $b \in \{0, 1\}$.

Given a common reference string $\text{crs} \leftarrow \text{Setup}(1^\lambda)$, a NIZK scheme is expected to satisfy the following properties:

Definition 2.17 (Correctness). For every $(x, w) \in R$, we have

$$\Pr[\text{Verify}(\text{crs}, x, \pi) = 1 \mid \pi \leftarrow \text{Prove}(\text{crs}, x, w)] = 1.$$

Definition 2.18 (Soundness). For every $x \notin L$, we have that:

$$\Pr[\exists \pi \text{ s.t. } \text{Verify}(\text{crs}, x, \pi) = 1] < \text{negl}(\lambda),$$

where the probability is taken over **Setup**.

Definition 2.19 (Straight-Line Extractable Knowledge Soundness). Let $\mathcal{D}$ be an auxiliary input distribution. There exists an extractor $\mathcal{E}$ such that for every prover algorithm $\tilde{\mathcal{P}}$ of size at most s ,

$$\Pr \left[\begin{array}{c} (x, w) \notin R \quad \wedge \\ \text{Verify}(\text{crs}, x, \pi) = 1 \end{array} \middle| \begin{array}{c} z \leftarrow \mathcal{D} \\ (x, \pi) \leftarrow \tilde{\mathcal{P}}(\text{crs}, z) \\ w \leftarrow \mathcal{E}(\text{crs}, x, \pi) \end{array} \right] \leq \kappa(\lambda, |x|, q, s)$$

and $\mathcal{E}$ runs in time at most $t(\lambda, |x|)$. We assume that the straight-line knowledge soundness error $\kappa(\lambda, |x|, q, s)$ is negligible in the security parameter λ .

⁴Here we allow the key generation randomness to be malicious. The condition can be explicitly written as $(\text{pk}_i, \text{sk}_i) \leftarrow \text{KeyGen}(\text{pp}; r)$ for **some** randomness r .

Straight-line extraction is possible in the random oracle model, the generic group model, or under a suitable knowledge assumption.

Definition 2.20 (Zero-Knowledge). There exists a PPT algorithm $\mathcal{S}$ such that for all $(x, w) \in R$, the following two distributions are computationally indistinguishable:

$$\{\text{Prove}(\text{crs}, x, w)\} \approx_c \{\mathcal{S}(\text{crs}, x)\}.$$

Remark 2.21. If needed, one can instantiate the NIZK with a zk-SNARK so that all the generated proofs are succinct.

2.6 Commitment

We assume a standard commitment scheme, definitions rephrased from [BS23].

Definition 2.22 (Commitment). A commitment scheme is a tuple of PPT algorithms

$$\text{Com} = (\text{Commit}, \text{Verify})$$

such that:

$\text{Commit}(m) \rightarrow (c, o)$: On input a message $m \in \mathcal{M}$, output a commitment c and an open string o .

$\text{Verify}(m, c, o) \rightarrow b$: On input a message $m \in \mathcal{M}$, a commitment c , and an opening o , outputs 1 if accept and 0 if reject.

We consider a commitment scheme that has both binding and hiding properties.

Definition 2.23 (Binding). For every PPT adversary $\mathcal{A}$ that outputs a 5-tuple (c, m_1, o_1, m_2, o_2) , the advantage

$$\Pr[m_1 \neq m_2 \wedge \text{Verify}(m_1, c, o_1) = \text{Verify}(m_2, c, o_2) = 1]$$

is negligible in the security parameter 1^λ .

Definition 2.24 (Hiding). Given a security parameter λ , the hiding game is defined as follows:

1. $\mathcal{A}$ chooses $m_0, m_1 \in \mathcal{M}$ and sends them to $\mathcal{C}$.
2. $\mathcal{C}$ chooses $b \leftarrow \{0, 1\}$, runs $(c, o) \leftarrow \text{Commit}(m_b)$, and sends c to $\mathcal{A}$.
3. $\mathcal{A}$ outputs a guess b' .

The advantage is defined as $|\Pr(b = b') - 1/2|$. The hiding property requires that for all PPT adversaries, the adversary has a negligible advantage.

The standard hiding property of a commitment implies a slightly stronger hiding, where the adversary can ask for multiple commitments.

Definition 2.25 (Multi-commitment Hiding). Given a security parameter λ , the (multi-commitment) hiding game for a commitment scheme Com is defined as follows:

1. $\mathcal{C}$ chooses $b \leftarrow \{0, 1\}$ and set $b_0 = b$ and $b_1 = 1 - b$.
2. $\mathcal{A}$ is allowed make polynomially many queries. For each query,
 - (a) $\mathcal{A}$ chooses $m_0, m_1 \in \mathcal{M}$ and sends them to $\mathcal{C}$.
 - (b) $\mathcal{C}$ runs $(c_i, o_i) \leftarrow \text{Com.Commit}(m_{b_i})$ for $i = 0, 1$, and sends (c_0, c_1) to $\mathcal{A}$.
3. $\mathcal{A}$ outputs a guess b' .

The advantage is defined as $|\Pr(b = b') - 1/2|$. The (multi-commitment) hiding property requires that for all PPT adversaries, the adversary has a negligible advantage.

3 One-round Blind Signature from MKFHE

In this section we present our first compiler that compiles a deterministic signature scheme S into a blind signature scheme. The compiler uses multi-key FHE (MKFHE) and a NIZK proof system. Let S be a deterministic signature scheme, let MKFHE be a multikey FHE scheme with threshold decryption, let NIZK be a non-interactive zero-knowledge that is straight-line extractable knowledge sound, and let Com be a hiding commitment scheme.

Construction 3.1. We construct a blind signature BS as follows:

$\text{BS.Setup}(1^\lambda) \rightarrow \text{pp}$: On input a security parameter λ , output

$$\text{pp} = (S.\text{Setup}(1^\lambda), \text{MKFHE}.\text{Setup}(1^\lambda), \text{NIZK}.\text{Setup}(1^\lambda)).$$

$\text{BS.KeyGen}(\text{pp}) \rightarrow (\text{pk}, \text{sk})$: On input a public parameter pp , the key generation algorithm is defined as follows:

1. Runs the signature key generation with respect to some randomness $r_{K,1}$.

$$(\text{pk}_S, \text{sk}_S) \leftarrow S.\text{KeyGen}(\text{pp}; r_{K,1}).$$

2. Runs the MKFHE key generation algorithm with respect to some randomness $r_{K,2}$.

$$(\text{pk}_{MS}, \text{sk}_{MS}) \leftarrow \text{MKFHE}.\text{KeyGen}(\text{pp}; r_{K,2}).$$

3. Encrypts the signing key via MKFHE and gets

$$\text{ct}_S \leftarrow \text{MKFHE}.\text{Encrypt}(\text{pk}_{MS}, \text{sk}_S; r_{K,3}).$$

4. Generates a commitment for the concatenation of sk_{MS} and sk_S .

$$(c_S, o_S) \leftarrow \text{Com}.\text{Commit}(\text{sk}_{MS} \parallel \text{sk}_S; r_{K,4}).$$

5. Proves the relation

$$R_K = \left\{ \begin{array}{l} (x, w) = \\ \left(\begin{array}{l} (\text{pk}_S, \text{pk}_{MS}, \text{ct}_S, c_S), \\ (\text{sk}_S, \text{sk}_{MS}, r_K, o_S) \end{array} \right) \end{array} \middle| \begin{array}{l} (\text{pk}_S, \text{sk}_S) = S.\text{KeyGen}(\text{pp}; r_{K,1}) \\ (\text{pk}_{MS}, \text{sk}_{MS}) = \text{MKFHE}.\text{KeyGen}(\text{pp}; r_{K,2}) \\ \text{ct}_S = \text{MKFHE}.\text{Encrypt}(\text{pk}_{MS}, \text{sk}_S; r_{K,3}) \\ (c_S, o_S) = \text{Com}.\text{Commit}(\text{sk}_{MS} \parallel \text{sk}_S; r_{K,4}) \end{array} \right\}$$

and get a NIZK proof

$$\pi_K \leftarrow \text{NIZK}.\text{Prove}(\text{crs}, (\text{pk}_S, \text{pk}_{MS}, \text{ct}_S, c_S), (\text{sk}_S, \text{sk}_{MS}, r_K, o_S)).$$

6. Outputs a pair $(\text{pk}, \text{sk}) = ((\text{pk}_S, \text{pk}_{MS}, \text{ct}_S, c_S, \pi_K), (\text{sk}_S, \text{sk}_{MS}, o_S))$

$\text{BS.User}_1(\text{pk}, m) \rightarrow (\text{st}, \text{umsg})$: On input a public key pk and a message $m \in \mathcal{M}$, the first user algorithm is defined as follows:

1. The user runs the verification algorithm

$$\text{NIZK}.\text{Verify}(\text{crs}, (\text{pk}_S, \text{pk}_{MS}, \text{ct}_S, c_S), \pi_K)$$

and outputs $\perp$ if the verification fails.

2. The user runs the MKFHE key generation algorithm and get

$$(\text{pk}_{\text{MU}}, \text{sk}_{\text{MU}}) \leftarrow \text{MKFHE.KeyGen}(\text{pp}; r_{\text{U},1})$$

for some randomness $r_{\text{U},1}$.

3. Using $\hat{\text{pk}} = (\text{pk}_{\text{MS}}, \text{pk}_{\text{MU}})$, the user expands ct_{S} to get

$$\hat{\text{ct}}_{\text{S}} \leftarrow \text{MKFHE.Expand}(\hat{\text{pk}}, \text{S}, \text{ct}_{\text{S}})$$

where S denotes the signer (and U denotes the user).

4. The user encrypts and expands the message $m \in \mathcal{M}$ and get

$$\text{ct}_m \leftarrow \text{MKFHE.Encrypt}(\text{pk}_{\text{MU}}, m; r_{\text{U},2})$$

$$\hat{\text{ct}}_m \leftarrow \text{MKFHE.Expand}(\hat{\text{pk}}, \text{U}, \text{ct}_m)$$

for some randomness $r_{\text{U},2}$. Now $\hat{\text{ct}}_m$ is the encryption of m under $\hat{\text{pk}}$.

5. The user generates a proof of valid key generation and encryption according to the relation

$$R_{\text{U}} = \left\{ \begin{array}{l} (x, w) = \\ ((\text{pk}_{\text{MU}}, \text{ct}_m), (r_{\text{U}}, \text{sk}_{\text{MU}}, m)) \end{array} \middle| \begin{array}{l} \text{MKFHE.KeyGen}(\text{pp}; r_{\text{U},1}) = (\text{pk}_{\text{MU}}, \text{sk}_{\text{MU}}) \\ \text{MKFHE.Encrypt}(\text{pk}_{\text{MU}}, m; r_{\text{U},2}) = \text{ct}_m \end{array} \right\}$$

where $r_{\text{U}} = (r_{\text{U},1}, r_{\text{U},2})$ and get a NIZK proof

$$\pi_{\text{U}} \leftarrow \text{NIZK.Prove}(\text{crs}, (\text{pk}_{\text{MU}}, \text{ct}_m), (r_{\text{U}}, \text{sk}_{\text{MU}}, m)).$$

6. The user runs the signing algorithm⁵ in an encrypted state and get

$$\hat{\text{ct}}_{\text{Eval}} \leftarrow \text{MKFHE.Eval}(\hat{\text{pk}}, \text{S.Sign}, \hat{\text{ct}}_{\text{S}}, \hat{\text{ct}}_m).$$

7. The user outputs $(\text{st}, \text{umsg}) = ((\text{sk}_{\text{MU}}, \hat{\text{ct}}_{\text{Eval}}), (\text{pk}_{\text{MU}}, \text{ct}_m, \pi_{\text{U}}))$.

BS.Signer(sk, umsg) $\rightarrow \text{smsg}$: On input a secret key sk and a user message umsg , the signer algorithm is defined as follows:

1. The signer runs the verification algorithm

$$\text{NIZK.Verify}(\text{crs}, (\text{pk}_{\text{MU}}, \text{ct}_m), \pi_{\text{U}})$$

and outputs $\perp$ if the verification fails.

2. The signer expands the ciphertexts $\text{ct}_{\text{S}}, \text{ct}_m$ and get

$$\hat{\text{ct}}_{\text{S}} \leftarrow \text{MKFHE.Expand}(\hat{\text{pk}}, \text{S}, \text{ct}_{\text{S}}), \quad \hat{\text{ct}}_m \leftarrow \text{MKFHE.Expand}(\hat{\text{pk}}, \text{U}, \text{ct}_m).$$

3. The signer evaluates the signing algorithm in an encrypted state and gets

$$\hat{\text{ct}}_{\text{Eval}} \leftarrow \text{MKFHE.Eval}(\hat{\text{pk}}, \text{S.Sign}, \hat{\text{ct}}_{\text{S}}, \hat{\text{ct}}_m).$$

4. The signer evaluates a partial decryption

$$p_{\text{S}} = \text{MKFHE.PartDec}(\text{sk}_{\text{MS}}, \hat{\text{ct}}_{\text{Eval}}; r_{\text{S}})$$

where r_{S} is a randomness.

⁵Note that this evaluation can be deferred to the second user algorithm, if desired.

5. The signer generates a proof for the relation

$$R_S = \left\{ \left(\begin{array}{l} (x, w) = \\ (p_S, \hat{ct}_{Eval}, c_S), \\ (sk_{MS}, r_S, o_S, sk_S, r_{K,4}) \end{array} \right) \middle| \begin{array}{l} (c_S, o_S) = \text{Com.Commit}(sk_{MS} \| sk_S; r_{K,4}) \\ p_S = \text{MKFHE.PartDec}(sk_{MS}, \hat{ct}_{Eval}; r_S) \end{array} \right\}$$

and get a NIZK proof

$$\pi_S \leftarrow \text{NIZK.Prove}(\text{crs}, (p_S, \hat{ct}_{Eval}, c_S), (sk_{MS}, r_S, o_S, sk_S, r_{K,4})).$$

6. The signer outputs $\text{msg} = (p_S, \pi_S)$.

$\text{User}_2(\text{st}, \text{msg}) \rightarrow \sigma$: On input a state st and a signer message msg , the second user algorithm is defined as follows:

1. The user checks the proof π_S of valid partial decryption and get

$$b \leftarrow \text{NIZK.Verify}(\text{crs}, (p_S, \hat{ct}_{Eval}, c_S), \pi_S).$$

If $b = 0$, outputs $\perp$.

2. The user evaluates a partial decryption and gets

$$p_U \leftarrow \text{MKFHE.PartDec}(sk_{MU}, \hat{ct}_{Eval}).$$

3. The user evaluates a final decryption and outputs a signature

$$\sigma \leftarrow \text{MKFHE.FinDec}(p_U, p_S).$$

$\text{BS.Verify}(\text{pk}, m, \sigma) \rightarrow b$: On input a public key pk , a message m , and a signature σ , output a bit

$$b = \text{S.Verify}(\text{pk}_S, m, \sigma).$$

This completes the description of the scheme. We stress that all the NIZK proofs are for relatively simple relations such as FHE key generation, encryption, and decryption. The bulk of the work is in the steps using FHE evaluation. Next, we prove its correctness and security properties.

Theorem 3.1 (Correctness). *The blind signature in Construction 3.1 is correct according to the correctness definition in Section 2.2, assuming that*

- S is correct.
- MKFHE is perfectly correct.
- NIZK is correct.

Proof. For an honestly generated blind signature σ , the output signature σ should satisfy

$$\sigma = \text{S.Sign}(sk_S, m)$$

and thus be verified as correct in the verification algorithm. Since all the evaluations are valid and the proofs are also valid, there is no failure ($\perp$). $\square$

To prove one-more unforgeability, we rely on the following lemma, which is essentially generating some additional materials on top of the standard signature unforgeability game. The proof is provided in Appendix B.

Lemma 3.2 (Unforgeability+). *Let MKFHE be an IND-CPA secure multikey FHE, S be an unforgeable signature scheme, and Com be a hiding commitment. Given public parameters generated by MKFHE.Setup(1^λ) and S.Setup(1^λ), consider the following game between a challenger and a PPT adversary:*

1. $\mathcal{C}$ runs $(\text{sk}_M, \text{pk}_M) \leftarrow \text{MKFHE.KeyGen}(\text{pp})$, $(\text{pk}_S, \text{sk}_S) \leftarrow \text{S.KeyGen}(\text{pp})$, $(c, o) \leftarrow \text{Com.Commit}(\text{sk}_M \parallel \text{sk}_S)$, and $\text{ct} \leftarrow \text{MKFHE.Encrypt}(\text{pk}_M, \text{sk}_S)$. $\mathcal{C}$ sends $\text{pk}_M, \text{pk}_S, \text{ct}$, and c to $\mathcal{A}$.
2. $\mathcal{A}$ generates $(\text{sk}'_M, \text{pk}'_M) \leftarrow \text{MKFHE.KeyGen}(\text{pp})$.
3. $\mathcal{A}$ queries $\mathcal{C}$ polynomially many times. Given the i -th message m_i , $\mathcal{C}$ computes $\sigma_i \leftarrow \text{Sign}(\text{sk}_S, m_i)$ and returns the signature σ_i to the adversary $\mathcal{A}$.
4. $\mathcal{A}$ outputs a forgery pair (m, σ) .
5. The adversary's advantage is defined as

$$\Pr [\text{S.Verify}(\text{pk}_S, m, \sigma) = 1 \wedge m \notin \{m_1, m_2, \dots\}].$$

Then, the advantage is negligible in the security parameter λ .

Theorem 3.3 (One-More Unforgeability). *The blind signature in Construction 3.1 satisfies one-more unforgeability (OMUF) according to the definition in Section 2.2, assuming that:*

- S satisfies unforgeability.
- MKFHE satisfies perfect correctness and simulation security.
- NIZK is straight-line extractable knowledge sound and zero-knowledge.
- Com is hiding.

Proof. We prove that a successful OMUF adversary to the blind signature BS gives an adversary to the attack game in Lemma 3.2. Suppose that there is an adversary $\mathcal{A}$ that breaks the one-more unforgeability. That is, in the game in Figure 1, the adversary has a non-negligible advantage:

$$\Pr [\text{OMUF}_{\text{BS}}^{\mathcal{A}}(\lambda) = 1] > \text{negl}(\lambda).$$

We consider an attack game in Lemma 3.2 between the challenger and the adversary $\mathcal{B}$. $\mathcal{B}$ takes the role of the OMUF challenger and interact with $\mathcal{A}$ as follows:

1. The Lemma 3.2 challenger provides the signing public key pk_S , the MKFHE public key pk_{MS} , a ciphertext ct_S that encrypts the signing key, and the commitment c_S to $\mathcal{B}$.
2. Since the tuple $(\text{pk}_S, \text{pk}_{MS}, \text{ct}_S, c_S)$ is correctly generated, it is in the language that corresponds to the proof π_K . By the zero-knowledge property, $\mathcal{B}$ simulates the proof.
3. $\mathcal{B}$ sends $(\text{pk}_S, \text{pk}_{MS}, \text{ct}_S, c_S, \pi_K)$ to $\mathcal{A}$, as if they were honestly generated from the blind signature key generation algorithm.
4. Every time $\mathcal{A}$ requests an oracle access to the oracle OSign in Figure 1,
 - (a) Using the straight-line knowledge extractor $\mathcal{E}$, $\mathcal{B}$ extracts the witnesses r_U , sk_{MU} , and m from π_U .
 - (b) $\mathcal{B}$ asks the Lemma 3.2 challenger to sign on the message m , getting the valid signature σ .

- (c) The (perfect) correctness of MKFHE ensures that applying partial decryptions and final decryption on ct_{Eval} should result in σ .
 - (d) Due to the simulation security of MKFHE, there exists a simulator $\mathcal{S}$ that simulates the partial decryption of $\hat{\text{ct}}_{\text{Eval}}$, giving $p_{\mathcal{S}}$ that is indistinguishable from the actual partial decryption.
 - (e) Due to the zero-knowledge property of NIZK, there exists a simulator $\mathcal{S}'$ that simulates the proof $\pi_{\mathcal{S}}$ which is indistinguishable from the actual proof.
 - (f) $\mathcal{B}$ sends $(p_{\mathcal{S}}, \pi_{\mathcal{S}})$ to $\mathcal{A}$.
5. Suppose that $\mathcal{A}$ accessed the oracle ℓ times with messages $m_1, \dots, m_{\ell}$ and provided a forgery $\{(m_i^*, \sigma_i^*)\}_{i \in [\ell+1]}$. There should exist one i such that $m_i^* \notin \{m_1, \dots, m_{\ell}\}$. For that m_i^* , we get the forgery σ_i^* , which is sent to the Lemma 3.2 challenger.

We constructed a valid Lemma 3.2 adversary using the OMUF adversary. This completes the proof. $\square$

Theorem 3.4 (Blindness). *The blind signature in Construction 3.1 satisfies the blindness property according to the definition in Section 2.2, assuming that:*

- NIZK is straight-line extractable knowledge sound and zero-knowledge.
- MKFHE satisfies perfect correctness and IND-CPA security.

Proof. We prove that a successful blindness adversary to the blind signature BS gives an adversary to the MKFHE IND-CPA game. Suppose that there is an adversary $\mathcal{A}$ that breaks the blindness. That is, in the game in Figure 2, the advantage

$$\text{Adv}_{\text{BS}}^{\text{blind}}(\mathcal{A}, \lambda) = \left| \Pr \left[\text{BLIND}_{\text{BS}}^{\mathcal{A}}(\lambda) = 1 \right] - \frac{1}{2} \right| > \text{negl}(\lambda).$$

We construct an adversary $\mathcal{B}$ to the MKFHE IND-CPA security game as follows:

1. $\mathcal{B}$ plays the role of the blindness challenger and honestly runs Setup.
2. $\mathcal{B}$ asks the IND-CPA challenger to provide pk_{MU} .
3. $\mathcal{B}$ asks the challenger to choose $b \leftarrow \{0, 1\}$.
4. Every time $\mathcal{A}$ requests access to the INIT oracle, $\mathcal{B}$ runs honestly.
5. For a request to the oracle OU_1 in Figure 2,
 - (a) $\mathcal{B}$ first verifies the proof π_{K} included in the public key and outputs $\perp$ if the verification fails.
 - (b) $\mathcal{B}$ extracts the witness from π_{K} using the straight-line extractor $\mathcal{E}$, getting

$$w = (\text{sk}_{\mathcal{S}}, \text{sk}_{\text{MS}}, r_{\text{K}}, o_{\mathcal{S}}).$$

- (c) $\mathcal{B}$ requests encryptions $\text{ct}_{m_{b_i}} = \text{Encrypt}(\text{pk}_{\text{MU}}, m_{b_i})$ (for $i = 0, 1$) to the IND-CPA challenger.
- (d) Since pk_{MU} and $\text{ct}_{m_{b_i}}$ are honestly generated, there exists a proof simulator $\pi'_{\text{U}} \leftarrow \mathcal{S}(\text{pk}_{\text{MU}}, \text{ct}_{m_{b_i}})$ that is indistinguishable from the valid proof π_{U} , due to the zero-knowledge property of NIZK.
- (e) $\mathcal{B}$ runs the computations to get $\hat{\text{ct}}_{\text{Eval}}$ and store it in st .
- (f) $\mathcal{B}$ sends $\text{umsg} = (\text{pk}_{\text{MU}}, \text{ct}_{m_{b_i}}, \pi'_{\text{U}})$ to $\mathcal{A}$.
- (g) If $\mathcal{A}$ later asks for an oracle access to $\text{OU}_1(1 - i)$, $\mathcal{B}$ uses the pregenerated ciphertext to answer the query.

6. For a request to the oracle OU_2 in Figure 2,
 - (a) $\mathcal{B}$ verifies the proof π_5 and outputs $\perp$ if the verification fails.
 - (b) $\mathcal{B}$ runs the signing algorithm on m_0, m_1 using sk_5 , and gets σ_0, σ_1 , respectively.
 - (c) The (perfect) correctness of MKFHE and the soundness of NIZK ensure that (σ_0, σ_1) actually equals to the results (but possibly with different orders) when we finish decrypting smsg 's.
 - (d) $\mathcal{B}$ sends (σ_0, σ_1) when requested.
7. As soon as $\mathcal{A}$ outputs the distinguishing bit b' , $\mathcal{B}$ outputs the same distinguishing bit.

From the adversary $\mathcal{A}$'s point of view, $\mathcal{B}$ is indistinguishable from the honest challenger, so it should have a non-negligible advantage. Hence, we can use $\mathcal{A}$ to build the MKFHE IND-CPA adversary. This completes the proof. $\square$

4 Committed Verifiable FHE

The construction in Section 3 provides a secure one-round blind signature. However, both parties must homomorphically construct the encrypted signature for themselves. In Section 5 and Appendix A, we construct blind signature schemes where only one side homomorphically computes the encrypted signature. To do so, we need a strong notion of verifiable FHE (VFHE) that we call CVFHE. In this section, we define and construct this notion.

4.1 Definition

Definition 4.1 (Committed Verifiable FHE). A committed verifiable FHE is a tuple of efficient algorithms

$$\text{CVFHE} = (\text{Setup}, \text{KeyGen}, \text{Commit}, \text{Encrypt}, \text{Eval}, \text{Decrypt}, \text{Verify})$$

such that:

Setup(1^λ) $\rightarrow$ **pp**: On input a security parameter λ , the parameter generation algorithm **Setup** outputs a string of public parameters **pp** which is a default input to all other algorithms.

KeyGen(**pp**) $\rightarrow$ (**pk**, **sk**): On input a public parameter **pp**, the key generation algorithm **KeyGen** outputs a public key **pk** and a secret key **sk**.

Commit($C; r$) $\rightarrow$ **com**: On input a private homomorphic circuit⁶ C and a randomness r , the commitment algorithm generates a commitment **com**.

Encrypt(**pk**, m) $\rightarrow$ **ct**: On input a message $m \in \mathcal{M}$ and a public key **pk**, the encryption algorithm **Encrypt** outputs a ciphertext **ct**.

Eval(**pk**, $(\text{ct}_i)_{1 \leq i \leq k}, C$) $\rightarrow$ (**ct'**, π): On input a private circuit $C \in \mathcal{M}_c$, a ciphertext tuple $\hat{\text{ct}} = (\text{ct}_1, \dots, \text{ct}_k)$, and a public key **pk**, the (possibly non-deterministic) evaluation algorithm generates a ciphertext **ct'** and a proof of evaluation π .

⁶A private homomorphic circuit refers to a homomorphic circuit that could possibly contain private inputs. For instance, the server that evaluates the homomorphic computations may have private inputs during the computation that should not be revealed to the secret key owner.

$\text{Decrypt}(\text{sk}, \text{ct}) \rightarrow m$: On input a secret key sk and a ciphertext ct , the decryption algorithm generates a message $m \in \mathcal{M}$.

$\text{Verify}(\text{sk}, \text{com}, (\text{ct}_i)_{1 \leq i \leq k}, \text{ct}', \pi) \rightarrow b$: On input a secret key sk , a commitment com , input ciphertexts $\text{ct}_1, \dots, \text{ct}_k$, an evaluated ciphertext ct' , and a proof of evaluation π , the verification algorithm outputs a verification bit b which output 1 for accept and 0 for reject.

A secure CVFHE scheme is expected to satisfy the following properties.

Definition 4.2 (Perfect Correctness). For a security parameter λ , any (private) circuit C , and plaintext inputs $m_1, \dots, m_k$, using the parameters $\text{pp} \leftarrow \text{Setup}(1^\lambda)$, and randomness distribution $\mathcal{D}$, it holds that

$$\Pr \left[\text{Decrypt}(\text{sk}, \text{ct}') = C(m_1, \dots, m_k) \mid \begin{array}{l} (\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\text{pp}) \\ r \leftarrow \mathcal{D}, \text{com} \leftarrow \text{Commit}(C; r) \\ \text{ct}_i \leftarrow \text{Encrypt}(\text{pk}, m_i) \text{ for } 1 \leq i \leq k \\ (\text{ct}', \pi) \leftarrow \text{Eval}(\text{pk}, (\text{ct}_i)_{1 \leq i \leq k}, C) \end{array} \right] = 1.$$

Definition 4.3 (Completeness). For a security parameter λ , any (private) circuit C , plaintext inputs $m_1, \dots, m_k$, using the parameters $\text{pp} \leftarrow \text{Setup}(1^\lambda)$, and randomness distribution $\mathcal{D}$, it holds that

$$\Pr \left[\text{Verify}(\text{sk}, \text{com}, (\text{ct}_i)_{1 \leq i \leq k}, \text{ct}', \pi) = 1 \mid \begin{array}{l} (\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\text{pp}) \\ r \leftarrow \mathcal{D}, \text{com} \leftarrow \text{Commit}(C; r) \\ \text{ct}_i \leftarrow \text{Encrypt}(\text{pk}, m_i) \text{ for } 1 \leq i \leq k \\ (\text{ct}', \pi) \leftarrow \text{Eval}(\text{pk}, (\text{ct}_i)_{1 \leq i \leq k}, C) \end{array} \right] = 1.$$

Definition 4.4 (Soundness). For a security parameter λ , any (private) circuit C , plaintext inputs $m_1, \dots, m_k$, adversary $\mathcal{A}$, using the parameters $\text{pp} \leftarrow \text{Setup}(1^\lambda)$, and randomness distribution $\mathcal{D}$, it holds that

$$\Pr \left[\begin{array}{l} \text{Verify}(\text{sk}, \text{com}, (\text{ct}_i)_{1 \leq i \leq k}, \text{ct}', \pi) = 1 \\ \wedge \\ \text{Decrypt}(\text{sk}, \text{ct}') \neq C(m_1, \dots, m_k) \end{array} \mid \begin{array}{l} (\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\text{pp}) \\ r \leftarrow \mathcal{D}, \text{com} \leftarrow \text{Commit}(C; r) \\ m_1, \dots, m_k \leftarrow \mathcal{A}_{\text{pp}}(\text{pk}) \\ \text{ct}_i \leftarrow \text{Encrypt}(\text{pk}, m_i) \text{ for } 1 \leq i \leq k \\ (\text{ct}', \pi) \leftarrow \mathcal{A}_{\text{pp}}((\text{ct}_i)_{1 \leq i \leq k}, \text{pk}) \end{array} \right] \leq \text{negl}(\lambda).$$

Definition 4.5 (IND-CPA Security). For a security parameter λ , for any PPT adversary $\mathcal{A}$, and using $\text{pp} \leftarrow \text{Setup}(1^\lambda)$, the adversary in the following attack game has a negligible advantage.

1. $\mathcal{C}$ runs $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\text{pp})$ and provides pk to $\mathcal{A}$.
2. $\mathcal{C}$ chooses $b \leftarrow \{0, 1\}$ and put $b_0 \leftarrow b$ and $b_1 \leftarrow 1 - b$.
3. Upon each query from $\mathcal{A}$,
 - (a) $\mathcal{A}$ chooses and sends $m_0, m_1 \in \mathcal{M}$ to $\mathcal{C}$.
 - (b) $\mathcal{C}$ provides $\text{Encrypt}(\text{pk}, m_{b_0})$, $\text{Encrypt}(\text{pk}, m_{b_1})$ to $\mathcal{A}$.
4. $\mathcal{A}$ outputs a guess b' . The experiment outputs 1 if $b = b'$.

In this game, the advantage is defined as $|\Pr[b' = b] - 1/2|$.

Definition 4.6 (Circuit Privacy). Given a security parameter λ and using $\text{pp} \leftarrow \text{Setup}(1^\lambda)$, let $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\text{pp})$, $m_1, \dots, m_k \in \mathcal{M}$, $C : \mathcal{M}^k \rightarrow \mathcal{M}$ be a circuit, and $\text{ct}_i \leftarrow \text{Enc}(\text{pk}, m_i)$ for each $i \in [k]$. We say that CVFHE satisfies **circuit privacy** if there exists a PPT simulator $\mathcal{S}$ such that

$$\begin{aligned} & (\text{Eval}(\text{pk}, C, \text{ct}_1, \dots, \text{ct}_k), \text{ct}_1, \dots, \text{ct}_k, \text{sk}, \text{pk}, 1^\lambda) \approx_S \\ & (\mathcal{S}(\text{pk}, C(m_1, \dots, m_k), \text{ct}_1, \dots, \text{ct}_k), \text{ct}_1, \dots, \text{ct}_k, \text{sk}, \text{pk}, 1^\lambda). \end{aligned}$$

Definition 4.7 (Hiding). Given a security parameter λ , the hiding game is defined as follows:

1. $\mathcal{A}$ chooses $m_0, m_1 \in \mathcal{M}_c$ and sends them to $\mathcal{C}$.
2. $\mathcal{C}$ chooses $b \leftarrow \{0, 1\}$, runs $\text{com} \leftarrow \text{Commit}(m_b)$, and sends c to $\mathcal{A}$.
3. $\mathcal{A}$ outputs a guess b' .

The advantage is defined as $|\Pr(b = b') - 1/2|$. The hiding property requires that for all PPT adversaries, the adversary has a negligible advantage.

Definition 4.8 (Multi-commitment Hiding). Given a security parameter λ , the (multi-commitment) hiding game for CVFHE is defined as follows:

1. $\mathcal{C}$ chooses $b \leftarrow \{0, 1\}$ and set $b_0 = b$ and $b_1 = 1 - b$.
2. $\mathcal{A}$ is allowed make polynomially many queries. For each query,
 - (a) $\mathcal{A}$ chooses $m_0, m_1 \in \mathcal{M}_c$ and sends them to $\mathcal{C}$.
 - (b) $\mathcal{C}$ runs $(c_i, o_i) \leftarrow \text{CVFHE.Commit}(m_{b_i})$ for $i = 0, 1$, and sends (c_0, c_1) to $\mathcal{A}$.
3. $\mathcal{A}$ outputs a guess b' .

The advantage is defined as $|\Pr(b = b') - 1/2|$. The (multi-commitment) hiding property requires that for all PPT adversaries, the adversary has a negligible advantage.

4.2 Construction

We construct CVFHE from FHE, NIZK, and commitment. We assume that FHE satisfies correctness, IND-CPA security, and circuit privacy. We assume that NIZK satisfies correctness, soundness, and zero-knowledge. We assume that commitment is binding and hiding.

Construction 4.1. We construct a CVFHE as follows:

Setup(1^λ) $\rightarrow$ **pp**: Run and output $\text{pp} \leftarrow (\text{FHE.Setup}(1^\lambda), \text{NIZK.Setup}(1^\lambda))$.

KeyGen(**pp**) $\rightarrow$ (**pk**, **sk**): Run and output $(\text{pk}_{\text{FHE}}, \text{sk}_{\text{FHE}}) \leftarrow \text{FHE.KeyGen}(\text{pp})$.

Commit($C; r$) $\rightarrow$ **com**: Run $(c, o) \leftarrow \text{Com.Commit}(C; r)$ and set $\text{com} = c$.

Encrypt(**pk**, m) $\rightarrow$ **ct**: Output $\text{ct} = \text{FHE.Encrypt}(\text{pk}_{\text{FHE}}, m)$.

Eval(**pk**, $(\text{ct}_i)_{1 \leq i \leq k}, C$) $\rightarrow$ (**ct'**, π): Run

$$\text{ct}' \leftarrow \text{FHE.ReRand}(\text{FHE.Eval}(\text{pk}_{\text{FHE}}, \text{ct}_1, \dots, \text{ct}_k, C); r').$$

Generate a proof according to the relation

$$R = \left\{ \left(\begin{array}{l} (x, w) = \\ (\text{ct}_1, \dots, \text{ct}_k, \text{ct}', \text{com}), \\ (C, r', o, r) \end{array} \right) \middle| \begin{array}{l} \text{ct}' = \text{FHE.ReRand}(\text{FHE.Eval}(\text{pk}_{\text{FHE}}, \text{ct}_1, \dots, \text{ct}_k, C); r') \\ (\text{com}, o) = \text{Com.Commit}(C; r) \end{array} \right\}$$

and get a NIZK proof

$$\pi \leftarrow \text{NIZK.Prove}(\text{crs}, (\text{ct}_1, \dots, \text{ct}_k, \text{ct}', \text{com}), (C, r', o, r)).$$

Output (ct', π) .

$\text{Decrypt}(\text{sk}, \text{ct}) \rightarrow m$: Run and output $m \leftarrow \text{FHE.Decrypt}(\text{sk}_{\text{FHE}}, \text{ct})$.

$\text{Verify}(\text{sk}, \text{com}, (\text{ct}_i)_{1 \leq i \leq k}, \text{ct}', \pi) \rightarrow b$: Run and output

$$b \leftarrow \text{NIZK.Verify}(\text{crs}, (\text{ct}_1, \dots, \text{ct}_k, \text{ct}', \text{com}), \pi).$$

Theorem 4.9. *The CVFHE defined in Construction 4.1 is perfectly correct.*

Proof. The perfect correctness of the underlying FHE directly gives the perfect correctness. $\square$

Theorem 4.10. *The CVFHE defined in Construction 4.1 is complete.*

Proof. The correctness of the underlying FHE and NIZK directly gives the completeness. $\square$

Theorem 4.11. *The CVFHE defined in Construction 4.1 is sound.*

Proof. Suppose that there exists a soundness adversary $\mathcal{A}$. We use this adversary to attack the soundness game of the underlying NIZK. We claim that the statement $x = (\text{ct}_1, \dots, \text{ct}_k, \text{ct}', \text{com})$ and the proof π with respect to the witness $w = (C, r', o, r)$ satisfies $\text{NIZK.Verify}(\text{crs}, x, \pi) = 1$ and $(x, w) \notin R$. Since

$$\text{CVFHE.Verify}(\text{sk}, \text{com}, (\text{ct}_i)_{1 \leq i \leq k}, \text{ct}', \pi) = 1,$$

it is clear that $\text{NIZK.Verify}(\text{crs}, x, \pi) = 1$. It remains to prove that $(x, w) \notin R$.

Suppose that $(x, w) \in R$. This means that the evaluation and the rerandomization was performed correctly with respect to the committed private circuit C and the randomness r' . In other words,

$$\text{ct}' = \text{FHE.ReRand}(\text{FHE.Eval}(\text{pk}_{\text{FHE}}, C, \text{ct}_1, \dots, \text{ct}_k); r')$$

where C is the committed circuit with respect to com . By the (perfect) correctness of FHE, we have that

$$\text{FHE.Decrypt}(\text{sk}_{\text{FHE}}, \text{ct}') = C(m_1, \dots, m_k).$$

This is a contradiction to $\text{CVFHE.Decrypt}(\text{sk}, \text{ct}') \neq C(m_1, \dots, m_k)$. This finishes the proof. $\square$

Theorem 4.12. *The CVFHE defined in Construction 4.1 satisfies IND-CPA.*

Proof. This is automatic from the IND-CPA security of the underlying FHE scheme FHE. $\square$

Theorem 4.13. *The CVFHE defined in Construction 4.1 satisfies circuit privacy.*

Proof. For simulating the ciphertext evaluation, we use the circuit privacy simulator $\mathcal{S}$ of the underlying FHE scheme FHE. It remains to simulate the evaluation proof. Since the simulated ciphertext ct' satisfies the relation for NIZK (because otherwise we can run the verification algorithm to distinguish the simulator from the real protocol⁷),

⁷Here we use the fact that NIZK is sound.

the zero-knowledge property of NIZK allows us to run a simulator S that generates the correct proof. This finishes the proof. $\square$

Theorem 4.14. *The CVPHE defined in Construction 4.1 is (multi-commitment) hiding.*

Proof. The (multi-commitment) hiding property of the underlying commitment scheme directly gives the (multi-commitment) hiding property of CVPHE. $\square$

5 Blind Signatures with Little Server Side Computation

We now turn to constructing a blind signature scheme where only the client needs to homomorphically compute the encrypted signature. The signer's work is greatly reduced compared to the scheme in Section 3. Our scheme uses a committed verifiable FHE (CVPHE) and a NIZK proof system.

Construction 5.1. We construct a blind signature BS as follows:

BS.Setup(1^λ) $\rightarrow$ **pp**: On input a security parameter λ , output

$$\text{pp} = (\text{S.Setup}(1^\lambda), \text{CVPHE.Setup}(1^\lambda), \text{NIZK.Setup}(1^\lambda)).$$

BS.KeyGen(**pp**) $\rightarrow$ (**pk**, **sk**): On input a public parameter **pp**, run

$$(\text{pk}_S, \text{sk}_S) \leftarrow \text{S.KeyGen}(\text{pp}; r_{K,1}), (\text{pk}_C, \text{sk}_C) \leftarrow \text{CVPHE.KeyGen}(\text{pp}; r_{K,2}),$$

$$\text{ct}_S \leftarrow \text{CVPHE.Encrypt}(\text{pk}_C, \text{sk}_S; r_{K,3}), (c, o) \leftarrow \text{Com.Commit}(\text{sk}_C \parallel \text{sk}_S; r_{K,4})$$

for some randomness $r_K = (r_{K,1}, r_{K,2}, r_{K,3}, r_{K,4})$. In addition, the signer generates a proof of the relation

$$R_K = \left\{ \begin{array}{l} (x, w) = \\ \left(\begin{array}{l} (\text{pk}_S, \text{pk}_C, \text{ct}_S, c), \\ (r_K, \text{sk}_S, \text{sk}_C, o) \end{array} \right) \end{array} \middle| \begin{array}{l} (\text{pk}_S, \text{sk}_S) \leftarrow \text{S.KeyGen}(\text{pp}; r_{K,1}) \\ (\text{pk}_C, \text{sk}_C) \leftarrow \text{CVPHE.KeyGen}(\text{pp}; r_{K,2}) \\ \text{ct}_S \leftarrow \text{CVPHE.Encrypt}(\text{pk}_C, \text{sk}_S; r_{K,3}) \\ (c, o) \leftarrow \text{Com.Commit}(\text{sk}_C \parallel \text{sk}_S; r_{K,4}) \end{array} \right\}$$

and get a NIZK proof

$$\pi_K \leftarrow \text{NIZK.Prove}(\text{crs}, (\text{pk}_S, \text{pk}_C, \text{ct}_S, c), (r_K, \text{sk}_S, \text{sk}_C, o)).$$

Output a pair

$$(\text{pk}, \text{sk}) = ((\text{pk}_S, \text{pk}_C, \text{ct}_S, c, \pi_K), (\text{sk}_S, \text{sk}_C)).$$

BS.User₁(**pk**, m) $\rightarrow$ (**st**, **umsg**): On input a public key **pk** and a message m , the first user algorithm is defined as follows:

1. The user verifies the proof π_K and outputs $\perp$ if verification fails.
2. The user samples a random message $k \in \mathcal{M}$ and runs the CVPHE commitment algorithm on the circuit $C = \text{S.Sign}(\cdot, m) \oplus k$ and get $\text{com} \leftarrow \text{CVPHE.Commit}(C)$. Here $\oplus$ refers to a group addition over $\mathcal{M}$, which serves as a one-time pad encryption by a key k .

3. The user proves the relation

$$R_U = \{(x, w) = (\text{com}, (m, k)) \mid \text{com} = \text{CVFHE.Commit}(\text{S.Sign}(\cdot, m) \oplus k)\}$$

and get a signature

$$\pi_C \leftarrow \text{NIZK.Prove}(\text{crs}, \text{com}, (m, k))$$

4. The user runs the evaluation algorithm and get

$$(\text{ct}, \pi_U) \leftarrow \text{CVFHE.Eval}(\text{pk}_C, \text{cts}, C).$$

5. The user outputs $\text{st} = (m)$ and

$$\text{umsg} = (\text{com}, \text{ct}, \pi_C, \pi_U).$$

BS.Signer(sk, umsg) $\rightarrow \text{smsg}$: On input a secret key sk and a user message umsg , the signer algorithm is defined as follows.

1. The signer runs the verification algorithms

$$\text{NIZK.Verify}(\text{crs}, \text{com}, \pi_C), \text{CVFHE.Verify}(\text{sk}, \text{com}, \text{cts}, \text{ct}, \pi_U)$$

and outputs $\perp$ if any one of them fails.

2. The signer runs the decryption algorithm and get

$$m' \leftarrow \text{CVFHE.Decrypt}(\text{sk}_C, \text{ct}; r_D)$$

for some randomness r_D .

3. The signer proves the relation

$$R_S = \left\{ \left(\begin{array}{l} (x, w) = \\ (\text{ct}, m', c), \\ (\text{sk}_C, \text{sk}_S, r_D, o, r_{K,4}) \end{array} \right) \middle| \begin{array}{l} m' = \text{CVFHE.Decrypt}(\text{sk}_C, \text{ct}; r_D) \\ (c, o) = \text{Com.Commit}(\text{sk}_C \parallel \text{sk}_S; r_{K,4}) \end{array} \right\}$$

and get a proof

$$\pi_D \leftarrow \text{NIZK.Prove}(\text{crs}, (\text{ct}, m', c), (\text{sk}_C, \text{sk}_S, r_D, o, r_{K,4})).$$

4. The signer outputs $\text{smsg} = (m', \pi_D)$.

BS.User₂(st, smsg) $\rightarrow \sigma$: On input a state st and a signer message smsg , the second user algorithm is defined as follows:

1. The signer first verifies the proof π_D and outputs $\perp$ if it fails.
2. The signer outputs $\sigma = m' \ominus k$.

BS.Verify(pk, m, σ) $\rightarrow b$: On input a public key pk , the message m , and a signature σ , the verification algorithm outputs

$$b \leftarrow \text{S.Verify}(\text{pk}_S, m, \sigma).$$

Our scheme satisfies correctness, one-more unforgeability, and blindness. The proofs are provided in Appendix B.

Theorem 5.1 (Correctness). *The blind signature defined in Construction 5.1 is correct, assuming that*

- *S is correct.*
- *CVFHE is perfectly correct and complete.*
- *NIZK is correct.*

Theorem 5.2 (One-More Unforgeability). *The blind signature defined in Construction 5.1 satisfies one-more unforgeability, assuming that*

- *S is correct and satisfies unforgeability.*
- *CVFHE is perfectly correct, complete, and IND-CPA secure.*
- *NIZK is correct, straight-line extractable knowledge sound, and zero-knowledge.*

Theorem 5.3 (Blindness). *The blind signature defined in Construction 5.1 satisfies blindness, assuming that*

- *S is correct.*
- *CVFHE is perfectly correct, complete, and circuit private.*
- *NIZK is correct, sound, and zero-knowledge.*

6 Extensions

We conclude with a few extensions to our framework. First we show how to relax the perfect correctness of the FHE to correctness with negligible failure probability. Second, we explain how to compile a signature into a threshold blind signature using both the universal thresholdizer and the universal blinder.

6.1 Relaxing Perfect Correctness

Although perfectly correct FHE is practical, its performance is not as good as the performance of FHE with negligible failure probability, mainly due to the worst case noise analysis during homomorphic evaluations. We discuss two constructions that work with the relaxed notion of correctness: evaluation correctness and the black box hash approach.

Recall that we mainly needed perfect correctness in the following subproblem.

Problem 6.1 (Homomorphic Signature Correctness). Given an FHE scheme FHE and a signature scheme S, consider the following problem:

- There is a signature $\sigma \leftarrow \text{S.Sign}(m)$.
- There are (possibly maliciously generated) an FHE key pair

$$(\text{pk}, \text{sk}) \leftarrow \text{FHE.KeyGen}(\text{pp})$$

and a ciphertext encrypting m $\text{ct} \leftarrow \text{FHE.Encrypt}(\text{pk}, m)$.

- There is an evaluated ciphertext $\text{ct}' \leftarrow \text{FHE.Eval}(\text{pk}, \text{S.Sign}, \text{ct})$.
- We need to prove that $\text{FHE.Decrypt}(\text{ct}') = \sigma$.

To solve this problem, we used NIZK to prove that the FHE keys and the ciphertext are correctly generated according to some randomness. The perfect correctness then guarantees that $\text{FHE.Decrypt}(\text{ct}') = \sigma$ as desired. If we should rely on the weaker correctness notion that allows a negligible failure probability, this solution is not valid:

There could exist some randomness such that evaluation correctness is not satisfied. That is, the evaluation can fail with nonnegligible probability.⁸

Solution 1: Evaluation Correctness.

The standard notion of (imperfect) correctness requires sampling keys and ciphertexts at random. We consider a stronger correctness notion where the evaluation correctness holds for any randomness for key generation and encryptions, but the evaluation can fail with negligible failure probability.

Definition 6.1 (Evaluation Correctness). For a security parameter λ , a circuit $C : \mathcal{M}^k \rightarrow \mathcal{M}$, messages $m_j \in \mathcal{M}$ for each $j \in [k]$, $\text{pp} \leftarrow \text{FHE.Setup}(1^\lambda)$, $(\text{pk}, \text{sk}) \leftarrow \text{FHE.KeyGen}(\text{pp}; r)$ for some randomness r , and $\text{ct}_j \leftarrow \text{FHE.Encrypt}(\text{pk}, m_j; r_j)$ for some randomness r_j each $j \in [k]$, it holds that:

$$\Pr \left[\begin{array}{l} \text{Decrypt}(\text{sk}, \text{ct}') \\ = C(m_1, \dots, m_k) \end{array} \middle| \begin{array}{l} \text{ct}' \leftarrow \text{FHE.Eval}(\text{pk}, C, \text{ct}_1, \dots, \text{ct}_k) \\ \text{ct}' \leftarrow \text{FHE.ReRand}(\text{ct}') \end{array} \right] = 1 - \text{negl}(\lambda).$$

In standard (exact) FHE instantiations, this can be achieved by enforcing the noise attached to public keys and ciphertexts to be sufficiently small. In addition to all the proofs that the keys and the ciphertext are correctly generated, the user additionally attaches the proof that the error is small. To get a small error, the user can either use rejection sampling or bounded noise distributions.

Solution 2: A Set-Avoiding Hash Function.

Recall the definition was weak correctness in FHE, where we allow a negligible failure probability.

Definition 6.2 (Weak Correctness). For a security parameter λ , a circuit $C : \mathcal{M}^k \rightarrow \mathcal{M}$, messages $m_j \in \mathcal{M}$ for each $j \in [k]$, and using $\text{pp} \leftarrow \text{FHE.Setup}(1^\lambda)$, it holds that:

$$\Pr \left[\begin{array}{l} \text{FHE.Decrypt}(\text{sk}, \text{ct}') \\ = C(m_1, \dots, m_k) \end{array} \middle| \begin{array}{l} (\text{pk}, \text{sk}) \leftarrow \text{FHE.KeyGen}(\text{pp}) \\ \forall j \in [k], \text{ct}_j \leftarrow \text{FHE.Encrypt}(\text{pk}, m_j) \\ \text{ct}' \leftarrow \text{FHE.Eval}(\text{pk}, C, \text{ct}_1, \dots, \text{ct}_k) \\ \text{ct}' \leftarrow \text{FHE.ReRand}(\text{ct}') \end{array} \right] = 1 - \text{negl}(\lambda).$$

For a security parameter λ , a circuit $C : \mathcal{M}^k \rightarrow \mathcal{M}$, messages $m_j \in \mathcal{M}$ for $j \in [k]$, using $\text{pp} \leftarrow \text{FHE.Setup}(1^\lambda)$, a randomness $r = (r_1, (r_{2j})_{j \in [k]})$, $(\text{pk}, \text{sk}) \leftarrow \text{FHE.KeyGen}(\text{pp}; r_1)$, $\text{ct}_j \leftarrow \text{FHE.Encrypt}(\text{pk}, m_j; r_{2j})$ for each $j \in [k]$, consider the probability

$$P((m_j)_{j \in [k]}, r) = \Pr \left[\begin{array}{l} \text{FHE.Decrypt}(\text{sk}, \text{ct}') \\ = C(m_1, \dots, m_k) \end{array} \middle| \begin{array}{l} \text{ct}' \leftarrow \text{FHE.Eval}(\text{pk}, C, \text{ct}_1, \dots, \text{ct}_k) \\ \text{ct}' \leftarrow \text{FHE.ReRand}(\text{ct}') \end{array} \right].$$

For a fixed set of messages $\vec{m} = (m_1, \dots, m_k)$, consider $P_{\vec{m}}(r) := P(\vec{m}, r)$. Let $\mathcal{T}$ be the set of all randomness strings, and $B_{\vec{m}}$ be the set of all randomness strings r such

⁸Note that this necessitates a commitment to the signing key, because the FHE encryption may not be binding. That is, there could exist two messages $m_1 \neq m_2$ such that

$$\text{FHE.Encrypt}(\text{pk}, m_1) = \text{FHE.Encrypt}(\text{pk}, m_2).$$

that $P_{\vec{m}}(r)$ has a nonnegligible failure probability. The failure probability across all r is at least

$$\frac{|B_{\vec{m}}|}{|\mathcal{T}|} \cdot (1 - P_{\vec{m}}(r))$$

and it is negligible due to the (weak) correctness property. Hence, we have that $|B_{\vec{m}}|/|\mathcal{T}| = \text{negl}(\lambda)$, which means that only a negligible fraction of the randomness strings are “bad.”

Definition 6.3. A hash function $H : \mathcal{S} \rightarrow \mathcal{T}$ is said to be **avoiding** if for every efficiently decidable set $\mathcal{T}' \subseteq \mathcal{T}$ of negligible size, and for every PPT adversary $\mathcal{A}_{\mathcal{T}'}$,

$$\Pr [H(s) \in \mathcal{T}' \mid s \leftarrow \mathcal{A}_{\mathcal{T}'}] = \text{negl}(\lambda).$$

This definition can be viewed as a single function analogue of output intractability [LV22, Definition 2.5], a special case of correlation intractability [CGH98]. In particular, for any efficiently decidable set $\mathcal{T}' \subseteq \mathcal{T}$ of negligible size, we may define $R_{\text{out}} = \mathcal{T}'$, and then our definition exactly corresponds to the output intractability for R_{out} .

If we assume the existence of such a hash function, and use a proof that the randomness string was generated as the output of this hash function, then we can prove security without perfect correctness. The solution to Problem 6.1 can be reformulated as follows:

- The adversary generates a key pair (pk, sk) and a ciphertext ct_m from the randomness generated by the hash function H . The adversary proves that these values were generated using the output of H on some seed chosen by the adversary.
- The avoiding property ensures that this set of keys and ciphertext has an evaluation correctness with negligible failure probability.
- By the evaluation correctness, we have that the resulting ciphertext after homomorphic signing decrypts to σ .

6.2 Threshold Blind Signatures

A threshold blind signature [VZK03, CKM⁺23, FNR25] is a combination of threshold signature and blind signature. One could understand it as a blind version of a threshold signature. Since [BGG⁺18] discusses compiling any signature scheme into a one-round threshold signature scheme, and we propose a compiler for blind signatures, it is a natural question to ask whether there exists a universal compiler for threshold blind signatures as well.

We give some informal constructions to this problem. The main idea is that the universal thresholdizer in [BGG⁺18] allows us to thresholdize any blind signature, including the one compiled through our universal blinder. To be explicit, the universal thresholdizer only needs to thresholdize the signer algorithm, and this automatically compiles any signature into a one-round threshold blind signature.

If we consider a t -out-of- n threshold blind signature, then the number of signer algorithms we should run is t . Hence, it is important to reduce the size of the signer computation in the scheme, and the construction in Section 5 has the smallest signer computation among the three constructions. In that case, the signing algorithm is an FHE decryption plus a proof of decryption. Note that we do not need to run FHE decryption or its proof homomorphically: We can instead redefine the signing algorithm as a threshold decryption plus the proofs of partial decryptions. The number of homomorphic signing, which is often the bottleneck, is 1 and is independent of the number of parties n or the threshold t . When t is fairly small, the user-side verifiable FHE

computation may be a bottleneck and one can use the MKFHE-based construction in Section 3. In this case, each signer sends the partial evaluation of the (homomorphic) signing algorithm, and the combination of partial evaluations becomes a partial decryption that the user requested. The remaining one with signer side verifiable computation is less interesting, as it involves t many verifiable homomorphic evaluations of the signing algorithm.

Acknowledgments. This work was partially supported by a grant from UBRI. The second author was supported by the Stanford Graduate Fellowship.

References

- [ABPS24] Shahla Atapoor, Karim Bagheri, Hilder V. L. Pereira, and Jannik Spiessens. Verifiable FHE via lattice-based SNARKs. *CiC*, 1(1):24, 2024.
- [AKSY21] Shweta Agrawal, Elena Kirshanova, Damien Stehle, and Anshu Yadav. Can round-optimal lattice-based blind signatures be practical? *Cryptology ePrint Archive*, Paper 2021/1565, 2021.
- [BCGY24] Foteini Baldimtsi, Jiaqi Cheng, Rishab Goyal, and Aayush Yadav. Non-interactive blind signatures: Post-quantum and stronger security. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part II*, volume 15485 of *LNCS*, pages 70–104. Springer, Singapore, December 2024.
- [BGG⁺18] Dan Boneh, Rosario Gennaro, Steven Goldfeder, Aayush Jain, Sam Kim, Peter M. R. Rasmussen, and Amit Sahai. Threshold cryptosystems from threshold fully homomorphic encryption. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part I*, volume 10991 of *LNCS*, pages 565–596. Springer, Cham, August 2018.
- [BK25] Dan Boneh and Jaehyung Kim. Homomorphic encryption for large integers from nested residue number systems. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part III*, volume 16002 of *LNCS*, pages 338–370. Springer, Cham, August 2025.
- [BLL⁺22] Fabrice Benhamouda, Tancrede Lepoint, Julian Loss, Michele Orrù, and Mariana Raykova. On the (in)security of ROS. *Journal of Cryptology*, 35(4):25, October 2022.
- [BNPS02] Mihir Bellare, Chanathip Namprempre, David Pointcheval, and Michael Se-manko. The power of RSA inversion oracles and the security of Chaum’s RSA-based blind signature scheme. In Paul F. Syverson, editor, *FC 2001*, volume 2339 of *LNCS*, pages 319–338. Springer, Berlin, Heidelberg, February 2002.
- [BNPS03] Mihir Bellare, Chanathip Namprempre, David Pointcheval, and Michael Se-manko. The one-more-RSA-inversion problems and the security of Chaum’s blind signature scheme. *Journal of Cryptology*, 16(3):185–215, June 2003.
- [Bol03] Alexandra Boldyreva. Threshold signatures, multisignatures and blind signatures based on the gap-Diffie-Hellman-group signature scheme. In Yvo Desmedt, editor, *PKC 2003*, volume 2567 of *LNCS*, pages 31–46. Springer, Berlin, Heidelberg, January 2003.

- [BS23] D. Boneh and V. Shoup. *A Graduate Course in Applied Cryptography (version 0.6)*. January 2023. Available at <https://toc.cryptobook.us/>.
- [CATZ24] Rutchathon Chairattana-Apirom, Stefano Tessaro, and Chenzhi Zhu. Pairing-free blind signatures from CDH assumptions. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part I*, volume 14920 of *LNCS*, pages 174–209. Springer, Cham, August 2024.
- [CDKS19] Hao Chen, Wei Dai, Miran Kim, and Yongsoo Song. Efficient multi-key homomorphic encryption with packed ciphertexts with application to oblivious neural network inference. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 395–412. ACM Press, November 2019.
- [CDVW24] Sofia Celi, Alex Davidson, Steven Valdez, and Christopher Wood. Privacy pass issuance protocols. RFC 9578, 2024.
- [CGH98] Ran Canetti, Oded Goldreich, and Shai Halevi. The random oracle methodology, revisited (preliminary version). In *30th ACM STOC*, pages 209–218. ACM Press, May 1998.
- [CGSY24] Alessandro Chiesa, Ziyi Guan, Shahar Samocha, and Eylon Yogev. Security bounds for proof-carrying data from straightline extractors. In Elette Boyle and Mohammad Mahmoody, editors, *TCC 2024, Part II*, volume 15365 of *LNCS*, pages 464–496. Springer, Cham, December 2024.
- [Cha82] David Chaum. Blind signatures for untraceable payments. In David Chaum, Ronald L. Rivest, and Alan T. Sherman, editors, *CRYPTO’82*, pages 199–203. Plenum Press, New York, USA, 1982.
- [CKM⁺23] Elizabeth C. Crites, Chelsea Komlo, Mary Maller, Stefano Tessaro, and Chenzhi Zhu. Snowblind: A threshold blind signature in pairing-free groups. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part I*, volume 14081 of *LNCS*, pages 710–742. Springer, Cham, August 2023.
- [EY24] Ehsan Ebrahimi and Anshu Yadav. Strongly secure universal thresholdizer. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part III*, volume 15486 of *LNCS*, pages 207–239. Springer, Singapore, December 2024.
- [FHKS16] Georg Fuchsbauer, Christian Hanser, Chethan Kamath, and Daniel Slamanig. Practical round-optimal blind signatures in the standard model from weaker assumptions. In Vassilis Zikas and Roberto De Prisco, editors, *SCN 16*, volume 9841 of *LNCS*, pages 391–408. Springer, Cham, August / September 2016.
- [FHS15] Georg Fuchsbauer, Christian Hanser, and Daniel Slamanig. Practical round-optimal blind signatures in the standard model. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *CRYPTO 2015, Part II*, volume 9216 of *LNCS*, pages 233–253. Springer, Berlin, Heidelberg, August 2015.
- [Fis06] Marc Fischlin. Round-optimal composable blind signatures in the common reference string model. In Cynthia Dwork, editor, *CRYPTO 2006*, volume 4117 of *LNCS*, pages 60–77. Springer, Berlin, Heidelberg, August 2006.
- [FNR25] Sebastian Faller, Guilhem Niot, and Michael Reichle. Lattice-based threshold blind signatures. Cryptology ePrint Archive, Paper 2025/1566, 2025.

- [FW24] Georg Fuchsbauer and Mathias Wolf. Concurrently secure blind Schnorr signatures. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part II*, volume 14652 of *LNCS*, pages 124–160. Springer, Cham, May 2024.
- [GOS06] Jens Groth, Rafail Ostrovsky, and Amit Sahai. Perfect non-interactive zero knowledge for NP. In Serge Vaudenay, editor, *EUROCRYPT 2006*, volume 4004 of *LNCS*, pages 339–358. Springer, Berlin, Heidelberg, May / June 2006.
- [GRS⁺11] Sanjam Garg, Vanishree Rao, Amit Sahai, Dominique Schröder, and Dominique Unruh. Round optimal blind signatures. In Phillip Rogaway, editor, *CRYPTO 2011*, volume 6841 of *LNCS*, pages 630–648. Springer, Berlin, Heidelberg, August 2011.
- [Hal17] Shai Halevi. *Homomorphic Encryption*, pages 219–276. Springer International Publishing, 2017.
- [Han23] Lucjan Hanzlik. Non-interactive blind signatures for random messages. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 722–752. Springer, Cham, April 2023.
- [KKL⁺23] Taechan Kim, Hyesun Kwak, Dongwon Lee, Jinyeong Seo, and Yongsoo Song. Asymptotically faster multi-key homomorphic encryption from homomorphic gadget decomposition. In Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda, editors, *ACM CCS 2023*, pages 726–740. ACM Press, November 2023.
- [KNYY21] Shuichi Katsumata, Ryo Nishimaki, Shota Yamada, and Takashi Yamakawa. Round-optimal blind signatures in the plain model from classical and quantum standard assumptions. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part I*, volume 12696 of *LNCS*, pages 404–434. Springer, Cham, October 2021.
- [KRS23] Shuichi Katsumata, Michael Reichle, and Yusuke Sakai. Practical round-optimal blind signatures in the ROM from standard assumptions. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part II*, volume 14439 of *LNCS*, pages 383–417. Springer, Singapore, December 2023.
- [KVMH24] C. Knabenhans, A. Viand, A. M.-G., and A. Hithnawi. vfhe: Verifiable fully homomorphic encryption. In *Proc. of the 12th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, pages 11–22. ACM, 2024.
- [LV22] Alex Lombardi and Vinod Vaikuntanathan. Correlation-intractable hash functions via shift-hiding. In Mark Braverman, editor, *ITCS 2022*, volume 215, pages 102:1–102:16. LIPIcs, January / February 2022.
- [MPD25] Jules Maire and Alan Pulval-Dady. Blind ECDSA from the ECDSA assumption. Cryptology ePrint Archive, Paper 2025/1827, 2025.
- [MW16] Pratyay Mukherjee and Daniel Wichs. Two round multiparty computation via multi-key FHE. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part II*, volume 9666 of *LNCS*, pages 735–763. Springer, Berlin, Heidelberg, May 2016.
- [PS96] David Pointcheval and Jacques Stern. Provably secure blind signature schemes. In Kwangjo Kim and Tsutomu Matsumoto, editors, *ASIACRYPT’96*, volume 1163 of *LNCS*, pages 252–265. Springer, Berlin, Heidelberg, November 1996.

- [QCY21] Xianrui Qin, Cailing Cai, and Tsz Hon Yuen. One-more unforgeability of blind ECDSA. In Elisa Bertino, Haya Shulman, and Michael Waidner, editors, *ESORICS 2021, Part II*, volume 12973 of *LNCS*, pages 313–331. Springer, Cham, October 2021.
- [SU11] Dominique Schröder and Dominique Unruh. Round optimal blind signatures. Cryptology ePrint Archive, Paper 2011/264, 2011.
- [VKH23] Alexander Viand, Christian Knabenhans, and Anwar Hithnawi. Verifiable fully homomorphic encryption, 2023.
- [VZK03] Duc Liem Vo, Fangguo Zhang, and Kwangjo Kim. A new threshold blind signature scheme from pairings. In *SCIS2003*, pages 233–238. SCIS, 2003.

A Blind Signatures with Little Client Side Computation

In this section, we compile a non-blind signature scheme into a blind signature scheme where only the signer computes the encrypted signature. To do so, we use the CVFHE from Section 4 and a NIZK proof system. This construction is closely related to two earlier constructions in the literature, one by Schröder and Unruh [SU11] and another by Agrawal et al. [AKSY21].

Construction A.1. We construct a blind signature BS as follows:

$\text{pp} \leftarrow \text{BS.Setup}(1^\lambda)$: On input a security parameter λ , output

$$\text{pp} = (\text{S.Setup}(1^\lambda), \text{CVFHE.Setup}(1^\lambda), \text{NIZK.Setup}(1^\lambda)).$$

$(\text{pk}, \text{sk}) \leftarrow \text{BS.KeyGen}(\text{pp})$: On input a public parameter pp , run $(\text{pk}_S, \text{sk}_S) \leftarrow \text{S.KeyGen}(\text{pp}; r_{K,1})$ and $\text{com} \leftarrow \text{CVFHE.Commit}(\text{S.Sign}(\text{sk}_S, \cdot); r_{K,2})$ for some randomness $r_K = (r_{K,1}, r_{K,2})$. In addition, the signer generates a proof π_K that these are generated correctly according to the randomness r_K :

$$\pi_K \leftarrow \text{NIZK.Prove}(\text{crs}, (\text{pk}_S, \text{com}), (r_K, \text{sk}_S, \text{S.Sign}(\text{sk}_S, \cdot))).$$

Output a pair

$$(\text{pk}, \text{sk}) = ((\text{pk}_S, \text{com}, \pi_K), \text{sk}_S).$$

$(\text{st}, \text{umsg}) \leftarrow \text{BS.User}_1(\text{pk}, m)$: On input a public key pk and a message m , the first user algorithm is defined as follows:

1. The user verifies the proof π_K and outputs $\perp$ if verification fails.
2. The user runs the CVFHE key generation algorithm and gets

$$(\text{sk}_C, \text{pk}_C) \leftarrow \text{CVFHE.KeyGen}(\text{pp}).$$

3. The user encrypts the message $m \in \mathcal{M}$ and get

$$\text{ct} \leftarrow \text{CVFHE.Encrypt}(\text{pk}_C, m).$$

4. The user generates a proof of valid encryption. That is, the user proves that there exists a secret witness sk_C , randomness $r_U = (r_{U,1}, r_{U,2})$, and message m , such that

- $\text{CVFHE.KeyGen}(\text{pp}; r_{U,1}) = (\text{sk}_C, \text{pk}_C)$.
- $\text{CVFHE.Encrypt}(\text{pk}_C, m; r_{U,2}) = \text{ct}$.

In terms of syntax,

$$\pi_U \leftarrow \text{NIZK.Prove}(\text{crs}, (\text{pk}_C, \text{ct}), (r_U, \text{sk}_C, m)).$$

5. The user outputs

$$\text{st} = (\text{sk}_C)$$

and

$$\text{umsg} = (\text{pk}_C, \text{ct}, \pi_U).$$

$\text{smsg} \leftarrow \text{BS.Signer}(\text{sk}, \text{umsg})$: On input a secret key sk and the user message umsg the signer algorithm is defined as follows.

1. The signer runs the verification algorithm

$$\text{NIZK.Verify}(\text{crs}, (\text{pk}_C, \text{ct}), \pi_U)$$

and output $\perp$ if the verification fails.

2. The signer runs the evaluation algorithm and get

$$(\text{ct}', \pi_S) \leftarrow \text{CVFHE.Eval}(\text{pk}_C, \text{ct}, \text{S.Sign}(\text{sk}_S, \cdot)).$$

3. The signer outputs $\text{smsg} = (\text{ct}', \pi_S)$.

$\sigma \leftarrow \text{BS.User}_2(\text{st}, \text{smsg})$: On input a state st and a signing message smsg , the second user algorithm is defined as follows:

1. The user runs the verification algorithm

$$\text{CVFHE.Verify}(\text{sk}_C, \text{com}, \text{ct}, \text{ct}', \pi_S)$$

and output $\perp$ if the verification fails.

2. The user runs the decryption algorithm and outputs

$$\sigma \leftarrow \text{CVFHE.Decrypt}(\text{sk}_C, \text{ct}').$$

$b \leftarrow \text{BS.Verify}(\text{pk}, m, \sigma)$: On input a public key pk , a message m , and a signature σ , the verification algorithm outputs

$$b \leftarrow \text{S.Verify}(\text{pk}_S, m, \sigma).$$

We prove correctness, one-more unforgeability, and blindness.

Theorem A.1. *The blind signature defined in Construction A.1 is correct, assuming that*

- S is correct.
- CVFHE is perfectly correct and complete.
- NIZK is correct.

Proof. Since CVFHE is (perfectly) correct and complete, and NIZK is correct, the homomorphic signing protocol outputs $\text{S.Sign}(\text{sk}_S, m)$ without failure. By the correctness of the underlying signature S , the verification algorithm verifies correctly. $\square$

Lemma A.2 (Unforgeability+). *Let CVFHE be a hiding CVFHE, and S be an unforgeable signature scheme. Given public parameters generated by $\text{CVFHE.Setup}(1^\lambda)$ and $\text{S.Setup}(1^\lambda)$, consider the following game between a challenger and a PPT adversary:*

1. $\mathcal{C}$ runs $(\text{pk}_S, \text{sk}_S) \leftarrow \text{S.KeyGen}(\text{pp})$ and $\text{com} \leftarrow \text{CVFHE.Commit}(\text{S.Sign}(\text{sk}_S, \cdot))$. $\mathcal{C}$ sends pk_S and com to $\mathcal{A}$.
2. $\mathcal{A}$ queries $\mathcal{C}$ polynomially many times. Given the i -th message m_i , $\mathcal{C}$ computes $\sigma_i \leftarrow \text{Sign}(\text{sk}_S, m_i)$ and returns the signature σ_i to the adversary $\mathcal{A}$.
3. $\mathcal{A}$ outputs a forgery pair (m, σ) .
4. The adversary's advantage is defined as

$$\Pr [\text{S.Verify}(\text{pk}_S, m, \sigma) = 1 \wedge m \notin \{m_1, m_2, \dots, \}].$$

Then, the advantage is negligible in the security parameter λ .

Proof. We consider the two games:

Game 0: Exactly the same as in the lemma statement.

Game 1: Exactly the same as Game 0, except that the challenger runs $\text{com} \leftarrow \text{CVFHE.Commit}(0)$ instead.

We claim that these two games are indistinguishable. To show this, we consider a game distinguishing adversary $\mathcal{A}$ and using this adversary to build an adversary $\mathcal{B}$ that attacks the hiding property of CVFHE. At the beginning, $\mathcal{B}$ plays the role of the game distinguishing game challenger and runs $(\text{pk}_S, \text{sk}_S) \leftarrow \text{S.KeyGen}(\text{pp})$. $\mathcal{B}$ then sets $m_0 = 0$ and $m_1 = C = \text{S.Sign}(\text{sk}_S, \cdot)$. Upon receiving the commitment com generated from $\text{CVFHE.Commit}(m_b)$, $\mathcal{B}$ can answer all the queries made by $\mathcal{A}$. At the end of the game, $\mathcal{A}$ outputs the guess bit b . Since $\mathcal{A}$ has a non-negligible advantage in the game distinguishing game, this breaks the hiding property of CVFHE.

In Game 1, S and CVFHE are completely independent. That is, now the commitment com is generated completely with CVFHE, and therefore Game 1 is exactly the same as the unforgeability game of S. This finishes the proof. $\square$

Theorem A.3. *The blind signature defined in Construction A.1 satisfies one-more unforgeability, assuming that*

- S is correct and satisfies unforgeability.
- CVFHE is perfectly correct, complete, circuit private, and hiding.
- NIZK is correct, straight-line extractable knowledge sound, and zero-knowledge.

Proof. Suppose that the blind signature does not satisfy the unforgeability property. We use this adversary $\mathcal{A}$ to construct an adversary $\mathcal{B}$ that attacks the game in Lemma A.2.

Once the Lemma A.2 challenger generates $(\text{pk}_S, \text{sk}_S) \leftarrow \text{S.KeyGen}(\text{pp})$ and $\text{com} \leftarrow \text{CVFHE.Commit}(\text{S.Sign}(\text{sk}_S, \cdot))$, the challenger sends $(\text{pk}_S, \text{com})$ to $\mathcal{B}$. Since the statement $(\text{pk}_S, \text{com})$ is in the language of the expected proof π_K , $\mathcal{B}$ runs the simulator and generate a simulated proof π'_K which is indistinguishable from a valid proof. $\mathcal{B}$ sends $\text{pk} = (\text{pk}_S, \text{com}, \pi'_K)$ to $\mathcal{A}$.

Every time $\mathcal{A}$ asks for an oracle access to the oracle OSign , $\mathcal{A}$ sends $(\text{pk}_C, \text{ct}, \pi_U)$ to $\mathcal{B}$. $\mathcal{B}$ verifies the proof and outputs $\perp$ if the verification fails. Successful verification

ensures that the key generation and encryption are correctly performed. $\mathcal{B}$ uses the straight-line extractor $\mathcal{E}$ of NIZK and get the witness

$$w = (r_U, \text{sk}_C, m).$$

$\mathcal{B}$ sends the message m to the Lemma A.2 challenger and asks for a signature σ . Since we know the correct evaluation of the circuit $C = \text{S.Sign}(\text{sk}_S, \cdot)$ at m , by the circuit privacy of CVFHE, we can run a simulator, getting a ciphertext and an evaluation proof (ct', π_S) . $\mathcal{B}$ sends $\text{smsg} = (\text{ct}', \pi_S)$ to $\mathcal{A}$. Since $\mathcal{B}$ interacted with $\mathcal{A}$ in a way that is indistinguishable from an honest challenger, $\mathcal{A}$ successfully forges the signature and sends it to $\mathcal{B}$. Among $\ell + 1$ signatures, there should be at least one message that was not included in the queries. $\mathcal{B}$ uses the forgery to forge the underlying signature. This finishes the proof. $\square$

Theorem A.4. *The blind signature defined in Construction A.1 satisfies blindness, assuming that*

- S is correct.
- CVFHE is perfectly correct, complete, and sound.
- NIZK is correct, straight-line extractable knowledge sound, and zero-knowledge.

Consider a blindness adversary $\mathcal{A}$ to the blind signature. We use $\mathcal{A}$ to construct an adversary $\mathcal{B}$ that attacks the CVFHE IND-CPA security. $\mathcal{B}$ plays the role of the blindness challenger and interacts with the IND-CPA challenger and $\mathcal{A}$ as follows:

1. $\mathcal{B}$ asks the IND-CPA challenger to choose $b \leftarrow \{0, 1\}$.
2. For each request to the init oracle, $\mathcal{B}$ honestly runs it.
3. For each request to OU_1 ,
 - (a) $\mathcal{B}$ verifies the proof π_K included in the public key and outputs $\perp$ if the verification fails.
 - (b) $\mathcal{B}$ extracts the witness from π_K using the straight-line extractor $\mathcal{E}$, getting
$$w = (r_K, \text{sk}_S, C).$$
 - (c) $\mathcal{B}$ requests encryptions $\text{ct}_{m_{b_i}} = \text{Encrypt}(\text{pk}_C, m_{b_i})$ for $i = 0, 1$ to the IND-CPA challenger.
 - (d) Since pk_C and $\text{ct}_{m_{b_i}}$ are honestly generated, there exists a proof simulation $\pi'_U \leftarrow \text{S}(\text{pk}_C, \text{ct}_{m_{b_i}})$ that is indistinguishable from a valid proof π_U , due to the zero-knowledge property of NIZK.
 - (e) $\mathcal{B}$ sends $\text{umsg} = (\text{pk}_C, \text{ct}_{m_{b_i}}, \pi'_U)$ to $\mathcal{A}$.
 - (f) If $\mathcal{A}$ later asks for an oracle access to $\text{OU}_1(1 - i)$, $\mathcal{B}$ uses the pregenerated ciphertext to answer the query.
4. For each request to OU_2 ,
 - (a) $\mathcal{B}$ verifies the proof π_S and outputs $\perp$ if the verification fails.
 - (b) $\mathcal{B}$ runs the signing algorithm on m_0, m_1 using sk_S , and gets σ_0, σ_1 respectively.
 - (c) The (perfect) correctness of CVFHE and NIZK ensures that (σ_0, σ_1) actually equals to the results (but possibly with different orders) when we decrypt the ciphertext ct' in smsg .

(d) $\mathcal{B}$ sends (σ_0, σ_1) when requested.

5. As soon as $\mathcal{A}$ outputs the distinguishing bit b' , $\mathcal{B}$ outputs the same distinguishing bits to the IND-CPA challenger.

From the adversary $\mathcal{A}$'s point of view, $\mathcal{B}$ is indistinguishable from the honest challenger, so it should have a non-negligible advantage. Hence, we can use $\mathcal{A}$ to build the CVPHE IND-CPA adversary. This finishes the proof.

B Deferred Proofs

Proof of Lemma 3.2.

Proof. We consider the following games.

Game 0: Exactly the game in the lemma statement.

Game 1: $\mathcal{C}$ runs $(c, o) \leftarrow \text{Com.Commit}(0)$ and provides c instead of a honest commitment. The rest remains the same as Game 0.

Game 2: $\mathcal{C}$ provides $\text{ct} = \text{MKFHE.Encrypt}(\text{pk}_M, 0)$ instead of a honest encryption. The rest remains the same as Game 1.

First, we prove that Game 0 and indistinguishable from Game 1. Suppose that there is an adversary $\mathcal{A}'$ that distinguishes Game 0 from Game 1. We use this adversary to construct an adversary $\mathcal{B}$ that breaks the hiding property of Com . In the beginning, $\mathcal{B}$ plays the role of the game distinguishing game challenger and runs $(\text{pk}_S, \text{sk}_S) \leftarrow \text{S.KeyGen}(\text{pp})$, $(\text{sk}_M, \text{pk}_M) \leftarrow \text{MKFHE.KeyGen}(\text{pp})$, $\text{ct} \leftarrow \text{MKFHE.Encrypt}(\text{pk}_M, \text{sk}_S)$. $\mathcal{B}$ then sets $m_0 = 0$ and $m_1 = \text{sk}_M \parallel \text{sk}_S$. Upon receiving the commitment c generated from $\text{Com.Commit}(m_b)$, $\mathcal{B}$ sends $\text{pk}_M, \text{pk}_S, \text{ct}$, and c to $\mathcal{A}'$. Since sk_S is available to $\mathcal{B}$, $\mathcal{B}$ can answer all the queries made by $\mathcal{A}'$. At the end of the game, $\mathcal{A}'$ outputs the guess bit b' that guesses b . Since $\mathcal{A}'$ has a non-negligible advantage in the game distinguishing game, this breaks the hiding property of the underlying commitment scheme, proving the indistinguishability.

Next, we prove that Game 1 is indistinguishable from Game 2 from the perspective of $\mathcal{A}$. Suppose that there is an adversary $\mathcal{A}'$ that distinguishes Game 1 from Game 2. We use this adversary to construct an adversary $\mathcal{B}$ that breaks the IND-CPA security of MKFHE. At the beginning, $\mathcal{B}$ plays the role of the Game distinguishing game challenger and generates $(\text{pk}_S, \text{sk}_S) \leftarrow \text{S.KeyGen}(\text{pp})$ and sets $m_0 = 0$ and $m_1 = \text{sk}_S$. Upon receiving pk_M and $\text{ct} = \text{MKFHE.Encrypt}(\text{pk}_M, m_b)$ from the MKFHE IND-CPA security challenger, $\mathcal{B}$ generates $(c, o) \leftarrow \text{Com.Commit}(0)$. $\mathcal{B}$ sends $\text{pk}_M, \text{pk}_S, \text{ct}$, and c to $\mathcal{A}'$. Since sk_S is available to $\mathcal{B}$, $\mathcal{B}$ can answer all the queries made by $\mathcal{A}'$. At the end of the game, $\mathcal{A}'$ outputs the guess bit b' that guesses b . Since $\mathcal{A}'$ has a non-negligible advantage in the game distinguishing game, this breaks the IND-CPA security, proving the indistinguishability.

Lastly, we prove that any adversary has a negligible advantage in Game 2. The key observation is that in Game 2, when compared to the unforgeability game of $\mathcal{S}$, provides $\text{pk}_M, \text{ct}, c$ in addition, which are completely orthogonal to $\mathcal{S}$. Hence, Game 2 is just equivalent to the unforgeability game. By the unforgeability of $\mathcal{S}$, this finishes the proof. $\square$

Proof of Theorem 5.1.

Proof. Since CVPHE is (perfectly) correct and complete, and NIZK is correct, the homomorphic signing protocol outputs $\text{S.Sign}(\text{sk}_S, m) \oplus k$ without failure. After decryption and subtracting the one-time pad, we get the desired signature σ . By the correctness of the underlying signature scheme $\mathcal{S}$, the verification verifies correctly. $\square$

We use the following lemma to prove Theorem 5.2.

Lemma B.1 (Unforgeability+). *Let CVFHE be an IND-CPA secure CVFHE, S be an unforgeable signature scheme, and Com be a hiding commitment. Given public parameters generated by CVFHE.Setup(1^λ) and S.Setup(1^λ), consider the following game between a challenger and a PPT adversary:*

1. $\mathcal{C}$ runs $(\text{sk}_\mathcal{C}, \text{pk}_\mathcal{C}) \leftarrow \text{CVFHE.KeyGen}(\text{pp})$, $(\text{pk}_\mathcal{S}, \text{sk}_\mathcal{S}) \leftarrow \text{S.KeyGen}(\text{pp})$, $(c, o) \leftarrow \text{Com.Commit}(\text{sk}_\mathcal{C} \parallel \text{sk}_\mathcal{S})$, and $\text{ct} \leftarrow \text{CVFHE.Encrypt}(\text{pk}_\mathcal{C}, \text{sk}_\mathcal{S})$. $\mathcal{C}$ sends $\text{pk}_\mathcal{C}, \text{pk}_\mathcal{S}, \text{ct}$, and c to $\mathcal{A}$.
2. $\mathcal{A}$ generates $(\text{sk}'_\mathcal{C}, \text{pk}'_\mathcal{C}) \leftarrow \text{CVFHE.KeyGen}(\text{pp})$.
3. $\mathcal{A}$ queries $\mathcal{C}$ polynomially many times. Given the i -th message m_i , $\mathcal{C}$ computes $\sigma_i \leftarrow \text{Sign}(\text{sk}_\mathcal{S}, m_i)$ and returns the signature σ_i to the adversary $\mathcal{A}$.
4. $\mathcal{A}$ outputs a forgery pair (m, σ) .
5. The adversary's advantage is defined as

$$\Pr [\text{S.Verify}(\text{pk}_\mathcal{S}, m, \sigma) = 1 \wedge m \notin \{m_1, m_2, \dots\}].$$

Then, the advantage is negligible in the security parameter λ .

Proof. The proof is essentially the same as Lemma 3.2. $\square$

Proof of Theorem 5.2.

Proof. We use an adversary $\mathcal{A}$ to the blind signature to construct an adversary $\mathcal{B}$ that attacks the game in Lemma B.1.

$\mathcal{B}$ first asks the Lemma B.1 challenger for public keys $\text{pk}_\mathcal{C}, \text{pk}_\mathcal{S}$, ciphertext $\text{ct}_\mathcal{S}$, and a commitment c . Since the tuple $(\text{pk}_\mathcal{S}, \text{pk}_\mathcal{C}, \text{ct}_\mathcal{S}, c)$ is a correctly generated statement for the proof $\pi_\mathcal{K}$, $\mathcal{B}$ can simulate the proof and output $\pi'_\mathcal{K}$ which is indistinguishable from the valid proof. $\mathcal{B}$ sends $\text{pk} = (\text{pk}_\mathcal{S}, \text{pk}_\mathcal{C}, \text{ct}_\mathcal{S}, c, \pi'_\mathcal{K})$ to $\mathcal{A}$.

Upon each request to the signing oracle OSign , $\mathcal{B}$ first checks the proofs $\pi_\mathcal{C}, \pi_\mathcal{U}$ and outputs $\perp$ if any verification fails. Otherwise, by using the straight-line extractor $\mathcal{E}$, $\mathcal{B}$ extracts the witness (m, k) . Due to the perfect correctness of CVFHE and due to the evaluation proof $\pi_\mathcal{U}$, we know that the decryption of ct equals to $\sigma \oplus k$, where σ is a signature signed on m . $\mathcal{B}$ asks the Lemma B.1 for a signature σ and compute $m' = \sigma \oplus k$. Since now we have a statement (ct, m', c) that is in the language of $R_\mathcal{S}$, we simulate the proof $\pi_\mathcal{D}$ via the zero-knowledge property and sends $(m', \pi_\mathcal{D})$ to $\mathcal{A}$.

In the adversary $\mathcal{A}$'s point of view, $\mathcal{B}$ is indistinguishable from the honest challenger and outputs a forgery $\{(m_i^*, \sigma_i^*)\}_{i \in [\ell+1]}$. There should exist at least one i such that $m_i^* \notin \{m_1, \dots, m_\ell\}$. $\mathcal{B}$ choose such a pair (m, σ) and sends it to the Lemma B.1 challenger. This gives an adversary to the game and finishes the proof. $\square$

Lemma B.2. *Let S be a signature scheme and CVFHE be a multi-commitment hiding CVFHE scheme. Consider the following game between a challenger and a PPT adversary:*

- $\mathcal{C}$ chooses a bit $b \leftarrow \{0, 1\}$ and set $b_0 = b, b_1 = 1 - b$.
- $\mathcal{A}$ asks polynomially many queries to $\mathcal{C}$. At each query,
 1. $\mathcal{A}$ runs $(\text{pk}, \text{sk}) \leftarrow \text{S.KeyGen}(\text{pp})$, chooses $m_0, m_1 \in \mathcal{M}$, and sends $(\text{pk}, \text{sk}, m_0, m_1)$ to $\mathcal{C}$.

2. $\mathcal{C}$ samples $k_0, k_1 \in \mathcal{M}$, runs $\sigma_i \leftarrow \text{S.Sign}(\text{sk}, m_i)$ for $i = 0, 1$, runs $\text{com}_i \leftarrow \text{CVFHE.Commit}(\text{S.Sign}(\cdot, m_{b_i}) \oplus k_i)$ for $i = 0, 1$, and sends $(\sigma_{b_0} \oplus k_0, \sigma_{b_1} \oplus k_1, \text{com}_0, \text{com}_1)$ to $\mathcal{A}$.

$\mathcal{A}$ outputs a distinguishing bit b' .

The advantage of $\mathcal{A}$, defined as $|\Pr[b' = b] - 1/2|$ is 0.

Proof. **Game 0:** Exactly the same game as in the lemma.

Game 1: Exactly the same game as Game 0 except that $\mathcal{C}$ additionally chooses random circuits C_0, C_1 and commits to them instead of committing to the signing circuits.

We first claim that Game 0 and Game 1 are indistinguishable from the adversary. Suppose that there exists a game-distinguishing adversary. Such an adversary can be used to build a multi-commitment hiding game adversary of CVFHE. Hence, this proves the indistinguishability.

In Game 1, the commitments do not give any additional information to the adversary because they are just random different commitments. Since k_0, k_1 are freshly chosen after every query, $\sigma_{b_0} \oplus k_0$ and $\sigma_{b_1} \oplus k_1$ serve as one-time pads and are truly random. Hence, the adversary's advantage should always be 0. This finishes the proof. $\square$

Proof of Theorem 5.3.

Proof. Consider a blindness adversary $\mathcal{A}$ to the blind signature. We use $\mathcal{A}$ to construct an adversary $\mathcal{B}$ that attacks the game in Lemma B.2. $\mathcal{B}$ plays the role of the blindness challenger and interacts with the Lemma B.2 challenger and $\mathcal{A}$ as follows:

1. $\mathcal{B}$ asks the Lemma B.2 challenger to choose $b \leftarrow \{0, 1\}$.
2. For each request to the init oracle, $\mathcal{B}$ honestly runs it.
3. For each request to OU_1 ,
 - (a) $\mathcal{E}$ verifies the proof π_K included in the public key and outputs $\perp$ if the verification fails.
 - (b) $\mathcal{B}$ extracts the witness using the straight-line extractor $\mathcal{E}$ and get

$$w = (r_K, \text{sk}_S, \text{sk}_C, o).$$

- (c) $\mathcal{B}$ sends $(\text{sk}_S, \text{pk}_S, m_0, m_1)$ to the Lemma B.2 challenger.
- (d) $\mathcal{B}$ receives $(\sigma_{b_0} \oplus k_0, \sigma_{b_1} \oplus k_1, \text{com}_0, \text{com}_1)$. Since $\mathcal{B}$ knows that the honestly run ciphertext decrypts to $\sigma_{b_i} + k_i$, $\mathcal{B}$ uses the circuit privacy simulator to get the pair (ct, π_U) . $\mathcal{B}$ sends $(\text{com}_i, \text{ct}, \pi_U)$ to $\mathcal{A}$.
- (e) If $\mathcal{B}$ later receives a request to OU_1 with index $1 - i$, $\mathcal{B}$ answers the query similarly.
4. For each request to OU_2 ,
 - (a) $\mathcal{B}$ first verifies the proof π_D and output $\perp$ if it fails.
 - (b) $\mathcal{B}$ generates (σ_0, σ_1) by running the signing algorithm on (m_0, m_1, sk_S) .

In the view of $\mathcal{A}$, $\mathcal{B}$ is indistinguishable from the honest challenger and thus $\mathcal{A}$ attacks the blindness, thereby attacking the game in Lemma B.2. This finishes the proof. $\square$